Redis & MySQL 面试八股背诵手册

面向 Go 后端 / AI Agent 开发实习

第一部分 Redis

一、基础与数据结构

二、持久化

三、过期删除与内存淘汰

四、高可用：主从、哨兵、集群

五、缓存问题

六、分布式锁

七、缓存一致性

八、其他常考

第二部分 MySQL

一、架构与 InnoDB

二、索引

三、事务与隔离级别

四、MVCC

五、锁

六、日志

七、SQL 优化

八、主从复制与分库分表

九、SQL 基础与表设计

第三部分 综合实战：Go 秒杀系统

一、设计文档：Seckill Service

二、代码实现

> 使用说明：按「主题 → 知识点」组织的结构化笔记，主体参考小林coding《图解Redis》《图解MySQL》及其面试题合集的知识框架，已重新整理改写。先记加粗关键词，再展开。涉及版本差异处已标注版本号。第三部分为 Redis + MySQL 综合实战（Go 秒杀系统设计文档与代码），其中【R-N】/【M-N】指向前文第 N 个知识点。

第一部分 Redis

一、基础与数据结构

1. Redis 高性能原因与线程模型

- **纯内存操作**：读写都在内存，避免磁盘 IO。
- **单线程执行命令**：避免多线程上下文切换和锁竞争，天然保证单条命令原子性。
- **IO 多路复用**：基于 epoll/kqueue 的 Reactor 模型，一个线程处理大量连接。
- **高效数据结构**：SDS、listpack、quicklist、跳表、渐进式 rehash 的 dict 等。
- 「单线程」指**命令执行（网络读写+命令处理）主流程是单线程**；持久化 fork 子进程、`UNLINK`/`lazyfree` 异步释放内存等由后台线程完成。
- **Redis 6.0 引入多线程 IO**（`io-threads`，默认关闭）：多线程只负责网络读写和协议解析，**命令执行仍是单线程**。

延伸 / 易错点

- **瓶颈**：通常是**内存和网络带宽**而非 CPU；大 key、`KEYS *`、高复杂度命令会阻塞主线程。
- **6.0 加多线程 IO 的原因**：高并发下网络读写占主线程时间多，多线程 IO 提升吞吐，命令执行仍无锁。

2. 网络模型与后台线程

- **单 Reactor 单线程（6.0 前）**：`epoll_create` 创建 epoll，监听 socket 注册连接事件 → 事件循环中 `epoll_wait` 等待事件：
  - 连接事件 → accept，为客户端 socket 注册**读事件**；
  - 读事件 → 读取请求、解析并执行命令，结果写入客户端发送缓冲区，客户端加入**待写队列**；
  - 每轮进入 `epoll_wait` 前（beforeSleep）先把待写队列的数据发出，发不完才注册**写事件**下轮继续发。
- **6.0 多 IO 线程**：主线程把读写 socket 的工作分发给 IO 线程并行处理，**命令仍由主线程串行执行**。
- **后台线程（BIO）**：关闭文件、AOF 刷盘（fsync）、lazyfree 异步释放内存（`UNLINK`、`FLUSHALL ASYNC`）三类耗时任务交给后台线程，避免阻塞主线程。

3. 数据类型与底层编码

类型	底层编码（以 Redis 7.x 为准）	典型场景
String	`int` / `embstr`（≤44 字节）/ `raw`，底层 SDS	缓存、计数器、分布式锁、Session
List	`quicklist`（双向链表，节点是 listpack；3.2~6.x 节点是 ziplist）	消息队列、最新列表
Hash	`listpack`（元素少且小，7.0 前为 ziplist）/ `hashtable`	对象存储、购物车
Set	`intset`（全整数且少）/ `listpack`（7.2+）/ `hashtable`	去重、共同好友、抽奖
ZSet	`listpack`（元素少，7.0 前为 ziplist）/ `skiplist + dict`	排行榜、延迟队列

- 扩展类型：**Bitmap**（签到、在线状态）、**HyperLogLog**（UV 基数统计，约 12KB，标准误差 0.81%）、**GEO**（附近的人，底层 ZSet + GeoHash）、**Stream**（5.0，消息队列，支持消费组和 ACK）。
- 编码转换阈值可配置，如 `hash-max-listpack-entries 128`、`hash-max-listpack-value 64`，超过即转为 hashtable，**只升级不降级**。

延伸 / 易错点

- **ZSet 同时用跳表 + dict**：dict 实现 **O(1) 按成员查分数**，跳表实现**按分数有序和范围查询**。
- **embstr vs raw**：embstr 将 redisObject 和 SDS **一次分配、内存连续**，只读；修改后转为 raw（两次分配）。

4. SDS 简单动态字符串

- **O(1) 获取长度**：记录 `len`。
- **二进制安全**：以 len 判断结束而非 `\0`，可存图片等任意数据。
- **杜绝缓冲区溢出**：修改前检查 `alloc` 空间，不够自动扩容。
- **减少内存重分配**：**空间预分配**（修改后长度 < 1MB 时分配同等大小的空闲空间，即翻倍；≥ 1MB 时额外多分配 1MB）+ **惰性空间释放**。
- 结尾仍保留 `\0`，可兼容部分 C 字符串函数。
- 3.2 后按长度使用 sdshdr8/16/32/64 不同头部，节省内存。

延伸 / 易错点

- **容量上限**：单个 String value 最大 **512MB**。

5. 跳表（ZSet 底层）

- 跳表 = **多层有序链表**，上层是下层的「索引」，查找从最高层开始逐层下降，平均 **O(log N)**。
- 新节点层数**随机生成**：每升一层概率 **p = 1/4**，最大层数 32（`ZSKIPLIST_MAXLEVEL`）。

- Redis 跳表节点额外有**后退指针**（支持逆序遍历）和每层的 **span 跨度**（用于 O(log N) 计算排名 `ZRANK`）。
- 选择跳表的原因：
  - **范围查询简单**：找到起点后沿底层链表遍历即可；红黑树需要中序遍历。
  - **实现简单、易调试**，插入删除只需改局部指针，无需旋转再平衡。
  - **内存灵活**：通过调整 p 平衡内存与性能。
  - 不用 B+ 树：B+ 树为**减少磁盘 IO** 设计，Redis 在内存中，没有这种需求。

延伸 / 易错点

- **插入流程**：自顶向下找每层前驱（记录 update[]）→ 随机层数 → 逐层插入并更新 span。

6. 哈希表 dict 与渐进式 rehash

- dict 内部有**两个哈希表 ht[0]、ht[1]**，链地址法解决冲突。
- 扩容触发（负载因子 = 元素数 / 桶数）：
  - 负载因子 **≥ 1** 且当前**没有执行 BGSAVE/BGREWRITEAOF**（避免 fork 期间写时复制放大内存）；
  - 负载因子 **≥ 5** 时强制扩容。
  - 负载因子 < 0.1 时缩容。
- **渐进式 rehash**：为 ht[1] 分配空间，`rehashidx` 从 0 开始；之后**每次增删改查顺带迁移一个桶**，另外定时任务（serverCron）也会分批迁移；完成后 ht[1] 变为 ht[0]。
- rehash 期间：**查找/删除/更新先查 ht[0] 再查 ht[1]，新增只写 ht[1]**。

延伸 / 易错点

- **渐进式的意义**：一次性迁移百万级 key 会长时间阻塞单线程主流程。

7. 压缩列表 ziplist 与 listpack

- ziplist 是连续内存的紧凑列表，每个 entry 记录**前一个节点长度 prevlen**（前节点 < 254 字节用 1 字节，否则 5 字节）。
- **连锁更新**：插入一个较大节点，后一节点 prevlen 从 1 字节变 5 字节，长度增加又可能导致再后一个节点也需扩展，最坏 $O(N^2)$。
- **listpack（5.0 引入，7.0 全面替代 ziplist）**：每个 entry **只记录自身长度**（encoding + data + backlen），不依赖前节点长度，**彻底消除连锁更新**；仍支持从后向前遍历。

延伸 / 易错点

- **小数据用紧凑结构的原因**：内存连续、无指针开销、CPU 缓存友好；元素少时 O(N) 遍历也很快。

二、持久化

8. RDB 快照

- RDB = 某一时刻内存数据的**二进制快照**。
- 触发：`SAVE`（主线程执行，阻塞，不推荐）、`BGSAVE`（**fork 子进程**生成）、配置 `save 900 1` 等自动触发、主从全量复制、`SHUTDOWN` 时。
- **写时复制（COW）**：fork 后父子进程共享物理内存页，父进程修改某页时才复制该页，子进程看到的是 fork 时刻的一致性快照。
- 优点：**文件紧凑、恢复速度快**，适合备份和全量复制。
- 缺点：**两次快照间的数据会丢失**；fork 本身会阻塞主线程（与页表大小相关），大内存实例 fork 耗时明显；写多时 COW 导致内存最多接近翻倍。

延伸 / 易错点

- **fork 阻塞点**：复制**页表**等进程数据结构，而非全部数据。

9. AOF 日志与重写

- AOF 记录**每条写命令**（文本协议），重启时重放恢复；**先执行命令再写日志**（不阻塞当前命令、无需语法检查，但可能丢失刚执行的命令）。
- 写回策略 `appendfsync`：
  - `always`：每条命令 fsync，最安全最慢；
  - `everysec`（默认）：每秒 fsync，**最多丢约 1 秒数据**；
  - `no`：由操作系统决定刷盘时机。
- **AOF 重写（BGREWRITEAOF）**：fork 子进程**根据当前内存数据**生成最简命令集（而非分析旧 AOF），压缩体积。重写期间新命令同时写入 AOF 缓冲和 **AOF 重写缓冲区**，子进程完成后追加并原子替换。
- **Redis 7.0 Multi-Part AOF**：AOF 拆为 base 文件 + 多个 incr 增量文件 + manifest 清单，重写期间新命令直接写新 incr 文件，**不再需要重写缓冲区**。

延伸 / 易错点

- **用子进程而非线程重写**：子进程有独立内存快照（COW），无需加锁即可保证数据一致。

10. 混合持久化与选型

- **混合持久化（4.0 引入，`aof-use-rdb-preamble yes`，5.0 起默认开启）**：AOF 重写时，前半部分写 **RDB 格式全量数据**，后半部分为重写期间的**增量 AOF 命令**。兼顾恢复速度和数据完整性。
- 选择：
  - 能容忍分钟级丢失、追求恢复快 → RDB；
  - 数据安全要求高 → AOF everysec；

    - 生产推荐 **RDB + AOF（混合持久化）**；纯缓存场景可都关闭。
- 重启加载优先级：**开启 AOF 时优先加载 AOF**（更完整）。

延伸 / 易错点

- **混合持久化缺点**：AOF 文件包含 RDB 二进制部分，可读性差，旧版本无法识别。

三、过期删除与内存淘汰

11. 过期删除策略

- 过期时间保存在**过期字典 expires**（key → 过期时间戳）。
- **惰性删除**：访问 key 时检查是否过期，过期则删除。CPU 友好，但不访问的过期 key 会占内存。
- **定期删除**：serverCron 周期执行（`hz` 默认 10，即每秒 10 次），每次**随机抽取一批设置了过期时间的 key**（经典描述为 20 个）删除其中已过期的；若过期比例超过阈值（经典描述 25%）则继续抽样，并有**执行时长上限**防止阻塞。新版本可通过 `active-expire-effort` 调整力度。
- Redis 采用 **惰性删除 + 定期删除** 组合。

延伸 / 易错点

- **不用定时删除的原因**：每个 key 一个定时器，大量 key 时 CPU 开销过大。
- **主从下的过期 key**：**从库不主动删除过期 key**，由主库删除后向从库同步 `DEL`；3.2 起从库读取已过期 key 返回空。

12. 内存淘汰策略（LRU / LFU）

- 达到 `maxmemory` 后触发，共 **8 种**：
    - `noeviction`（**默认**）：不淘汰，写命令报错；
    - 所有 key 中：`allkeys-lru`、`allkeys-lfu`、`allkeys-random`；
    - 设置了过期时间的 key 中：`volatile-lru`、`volatile-lfu`、`volatile-random`、`volatile-ttl`（优先淘汰剩余寿命最短的）。
- **近似 LRU**：redisObject 中 24 bit 记录最近访问时间；淘汰时**随机采样 `maxmemory-samples`（默认 5）个 key**，淘汰最久未访问的（3.0 起有候选淘汰池优化）。不维护全局链表，节省内存和开销。
- **LFU（4.0 引入）**：同样 24 bit 拆分为 **16 bit 最近递减时间 + 8 bit 对数访问计数器**（最大 255，概率递增）；计数会随时间衰减（`lfu-decay-time`），解决 LRU 「偶尔访问一次就被当成热点」的问题。

延伸 / 易错点

- **选型**：缓存场景通常 `allkeys-lru`；热点明显、访问频率差异大时选 `allkeys-lfu`。

四、高可用：主从、哨兵、集群

13. 主从复制：全量与增量

- **全量复制**：
  1. 从库发送 `PSYNC ? -1`（首次，不知道主库 replid）；
  2. 主库回复 `FULLRESYNC replid offset`，执行 **BGSAVE** 生成 RDB 发送给从库；
  3. 期间新写命令写入该从库的 **replication buffer**；
  4. 从库清空旧数据、加载 RDB，主库再发送缓冲中的命令。
- **命令传播**：之后主库将写命令持续异步发送给从库，基于**长连接**。
- **增量复制（2.8 起 PSYNC）**：断线重连后从库发送 `PSYNC replid offset`；主库在**复制积压缓冲区 repl_backlog_buffer（环形缓冲区）** 中查找，offset 仍在缓冲区内则回复 `CONTINUE` 只发送缺失部分，否则退化为全量复制。
- Redis 复制默认**异步**，主库不等待从库确认。

延伸 / 易错点

- **避免频繁全量复制**：调大 `repl-backlog-size`（建议 ≥ 写入速率 × 平均断线时长 × 2）。
- **主从级联**：「主-从-从」结构减轻主库 fork 和网络压力。

14. 哨兵 Sentinel 与故障转移

- 作用：**监控、自动故障转移、通知、配置提供**（客户端通过哨兵获取主库地址）。
- **主观下线（SDOWN）**：单个哨兵 PING 超过 `down-after-milliseconds` 未收到有效回复。
- **客观下线（ODOWN）**：询问其他哨兵，认为下线的数量 ≥ **quorum** 时判定（只针对主库）。
- **选举 Leader 哨兵**：类 Raft，获得**半数以上**哨兵投票且票数 ≥ quorum 才能成为 Leader，执行故障转移。因此哨兵至少部署 **3 个**。
- **选新主库**：过滤掉下线/网络不佳的从库 → `replica-priority` 小者优先 → **复制偏移量最大**者优先 → **runid 最小**者。
- 执行：新主 `REPLICAOF NO ONE`，其余从库指向新主，旧主恢复后变为从库，通过发布订阅通知客户端。

延伸 / 易错点

- **哨兵互相发现**：通过主库的 `__sentinel__:hello` 频道发布订阅；通过 `INFO` 命令从主库获取从库列表。

15. Redis Cluster 与哈希槽

- **去中心化**分片集群，数据分到 **16384 个哈希槽**：`slot = CRC16(key) mod 16384`，每个主节点负责一部分槽，每个主节点可有从节点。
- 节点之间通过 **Gossip 协议**交换状态（PING/PONG/MEET/FAIL）。

- **客户端重定向**：
  - `MOVED`：槽已**永久**迁移到其他节点，客户端更新本地槽映射；
  - `ASK`：槽**迁移中**，仅本次请求转向目标节点（先发 `ASKING`）。
- **Hash Tag**：`{user1}:a` 与 `{user1}:b` 只按 `{}` 内计算 slot，保证落在同一节点，用于多 key 操作/Lua 脚本。
- 故障转移：节点间互相检测 PFAIL → 多数主节点确认 FAIL → 从节点发起选举，**获得多数主节点投票**后升级。
- 为什么 16384：心跳包携带槽位图 **16384 bit = 2KB**，65536 则需 8KB，带宽开销大；作者建议集群主节点**不超过 1000 个**，16384 槽足够。

延伸 / 易错点

- **集群限制**：跨 slot 的多 key 操作（MGET、事务、Lua）不支持（报 CROSSSLOT）；只能使用 db0。
- **哈希槽 vs 一致性哈希**：槽是固定逻辑分区，迁移以槽为单位，扩缩容和数据分布更可控。

16. 脑裂与数据丢失

- 主库因网络分区与哨兵/从库失联但仍在运行，哨兵选出新主后出现**两个主库**；旧主恢复后降为从库并**全量同步新主数据，期间写入旧主的数据丢失**。
- 缓解配置：
  - `min-replicas-to-write N`：至少 N 个从库连接正常才接受写；
  - `min-replicas-max-lag T`：从库复制延迟不超过 T 秒。
- 本质是 Redis 主从异步复制，**无法完全避免数据丢失**，需要强一致应使用 etcd/ZooKeeper 等共识系统。

五、缓存问题

17. 缓存穿透与布隆过滤器

- 定义：请求的数据**在缓存和数据库中都不存在**，每次都打到数据库（常见于恶意攻击）。
- 方案：
  - **参数校验**：拦截非法 ID；
  - **缓存空值**：数据库查不到也缓存空对象，设置**较短 TTL**；
  - **布隆过滤器**：先判断 key 是否可能存在，不存在直接返回。

延伸 / 易错点

- **布隆过滤器原理**：bit 数组 + k 个哈希函数，插入时置 k 位为 1；查询时任一位为 0 则**一定不存在**，全为 1 则**可能存在**（有误判）。**不支持删除**（可用计数布隆过滤器/布谷鸟过滤器）。
- **误判率**：与 bit 数组大小 m、哈希函数个数 k、元素数 n 有关；m 越大误判越低。

18. 缓存击穿

- 定义：某个**热点 key 过期**瞬间，大量并发请求同时打到数据库。
- 方案：
  - **互斥锁/单飞**：只允许一个请求查库回填，其余等待或重试（Go 中可用 `golang.org/x/sync/singleflight`）；
  - **逻辑过期**：缓存不设物理 TTL，value 中存过期时间，发现过期后异步重建，期间返回旧值；
  - **热点 key 不过期** / 提前续期。

延伸 / 易错点

- **取舍**：互斥锁保证一致性但有等待；逻辑过期可用性高但短时间返回旧数据。

19. 缓存雪崩

- 定义：**大量 key 同时过期**或 **Redis 宕机**，请求集中打到数据库。
- 方案：
  - 过期时间加**随机值**打散；
  - **高可用**：主从 + 哨兵 / Cluster；
  - **多级缓存**：本地缓存（如 Go 的 freecache/bigcache、Java 的 Caffeine）+ Redis；
  - **限流、熔断、降级**保护数据库；
  - 缓存预热。

延伸 / 易错点

- **三者区分**：穿透 = 数据不存在；击穿 = 单个热点 key 失效；雪崩 = 大面积失效或服务宕机。

20. 大 key 与热 key

- **大 key**：value 过大（如 String > 10KB、集合元素数万以上，具体阈值依业务）。危害：阻塞主线程、网络拥塞、内存不均、删除时阻塞。
  - 发现：`redis-cli --bigkeys`、`MEMORY USAGE key`、离线分析 RDB。
  - 处理：**拆分**（按字段/分片拆成多个 key）、压缩、使用 **`UNLINK` 异步删除**（4.0+），开启 `lazyfree-lazy-*`。
- **热 key**：访问频率极高，单节点压力过大。
  - 发现：`redis-cli --hotkeys`（需 LFU 策略）、客户端/代理层统计。
  - 处理：**本地缓存**、**热 key 复制多份**（加后缀分散到不同节点）、读写分离。

21. KEYS 与 SCAN

- **KEYS pattern**：一次遍历全部 key，O(N)，**会阻塞主线程**，生产环境禁用（可 `rename-command` 屏蔽）。

- **SCAN cursor [MATCH] [COUNT]**：基于游标**分批增量迭代**，每次只返回一小批，不长时间阻塞。
  - COUNT 只是**提示**，每次返回数量不固定；游标返回 0 表示遍历结束。
  - **保证**：完整遍历期间**一直存在**的 key 一定会被返回；**可能返回重复 key**，需客户端去重；遍历中新增/删除的 key 可能出现也可能不出现。
  - 采用**高位进位（反向二进制）**的游标顺序，使扩容/缩容 rehash 时也不会漏掉已存在的 key。
- 集合类型对应有 `HSCAN`、`SSCAN`、`ZSCAN`。

22. Redis 变慢排查思路

- **慢命令**：`SLOWLOG GET` 查看慢日志（阈值 `slowlog-log-slower-than`）；排查 O(N) 命令、大 key 操作。
- **大 key**：读写和删除大 key 耗时长 → 拆分、`UNLINK`。
- **集中过期**：大量 key 同一时刻过期，主动过期清理占用主线程 → 过期时间加随机值。
- **内存达到上限**：每次写入前都要淘汰 key → 调整 maxmemory、淘汰策略，拆分实例。
- **fork 耗时**：`INFO` 中 `latest_fork_usec`；控制实例内存大小，避开高峰做持久化。
- **AOF 刷盘**：`appendfsync always` 或磁盘 IO 压力大 → everysec，`no-appendfsync-on-rewrite yes`。
- **操作系统**：内存 swap、透明大页（THP，COW 时复制 2MB 大页）、CPU 绑核不当、网络带宽打满。
- 工具：`redis-cli --latency`、`LATENCY DOCTOR`（需开启 latency-monitor）。

六、分布式锁

23. Redis 分布式锁

- 加锁：`SET lock_key <唯一值uuid> NX PX 30000` —— **NX 保证互斥，PX 设置过期防止死锁，二者一条命令保证原子性**。
- 解锁：用 **Lua 脚本**先比较 value 是否为自己的 uuid 再 `DEL`，保证「判断+删除」原子，**防止误删他人的锁**。

```lua
if redis.call("get", KEYS[1]) == ARGV[1] then
  return redis.call("del", KEYS[1])
else
  return 0
end
```

- 常见坑：
  - **锁过期但业务未完成** → 自动续期（看门狗）；

- **不可重入** → 用 Hash 结构记录持有者和重入次数；
- **主从切换锁丢失**：主库加锁后未同步到从库即宕机，新主上没有锁 → Redlock 或改用强一致组件。

延伸 / 易错点

- **Redisson 看门狗**：未指定 leaseTime 时，默认锁超时 **30 秒**，后台每 **10 秒（超时的 1/3）** 续期；客户端宕机则不再续期，锁自然过期。
- **Redlock**：向 **N（通常 5）个相互独立的 Redis 主节点**依次加锁，**超过半数（≥ N/2+1）成功且总耗时小于锁有效期**才算成功，失败则向所有节点释放。
- **Redlock 争议**：Martin Kleppmann 指出其依赖时钟假设，GC 停顿/时钟跳变下仍可能不安全，建议使用 **fencing token（单调递增令牌）** 由存储端校验。

24. Redis / ZooKeeper / etcd 锁对比

- **Redis**：性能高，基于过期时间，AP 倾向，极端情况可能失效。
- **ZooKeeper**：**临时顺序节点** + Watch 前一个节点，会话断开节点自动删除；CP，可靠性高但性能较低。
- **etcd**：基于 **Raft** 强一致，使用 **Lease（租约）+ Revision**，按 revision 最小者获得锁，Watch 前一个 key；Go 生态常用（`clientv3/concurrency`）。
- 选择：允许极少数失效、追求性能 → Redis；要求正确性 → etcd/ZooKeeper。

七、缓存一致性

25. Cache Aside 与缓存一致性

- 最常用 **Cache Aside 旁路缓存**：
  - 读：先读缓存，命中返回；未命中读数据库，再写入缓存；
  - 写：**先更新数据库，再删除缓存**。
- **为什么删除缓存而不是更新缓存？** —— 并发写时更新缓存的顺序可能与数据库不一致；且缓存值可能需复杂计算、写多读少时更新是浪费。删除后由下次读懒加载。
- **为什么先更新库再删缓存，而不是先删缓存？** —— 先删缓存时，另一读请求在库更新前读到旧值并回填缓存，导致长期脏数据。先更新库的异常窗口要求「读请求在缓存失效时读到旧值，并在写请求删缓存**之后**才回填」，由于缓存写入远快于数据库写入，概率很低。
- **删除缓存失败怎么办？** —— **重试**：将删除操作投递到**消息队列**重试；或**订阅 binlog**（Canal 等）异步删除缓存，与业务解耦。
- 兜底：缓存**设置过期时间**，保证最终一致。

延伸 / 易错点

- **延迟双删**：先删缓存 → 更新数据库 → **sleep 一段时间（大于一次读请求耗时 + 主从延迟）再删一次**，用于「先删缓存」方案或读写分离从库延迟场景；缺点是延迟时间难确定。

- **强一致**：缓存 + 数据库很难强一致，需分布式读写锁等，牺牲性能；一般追求**最终一致性**。

26. 其他缓存读写模式

- **Read/Write Through**：应用只和缓存交互，由缓存层同步读写数据库（需缓存组件支持）。
- **Write Behind（Write Back）**：只写缓存，**异步批量**写回数据库；性能高，但宕机可能丢数据，一致性弱（类似 CPU Cache 写回、MySQL Buffer Pool）。

八、其他常考

27. 事务、Lua 脚本与 Pipeline

- **事务**：`MULTI` 开启 → 命令入队 → `EXEC` 执行；`WATCH` 实现乐观锁（被监视 key 变化则 EXEC 返回空）。
  - 保证**顺序执行、不被打断**（隔离）；**不支持回滚**：运行时某条出错，其他命令照常执行；入队时语法错误则整个事务不执行。
- **Lua 脚本**：`EVAL`，整个脚本**原子执行**，可包含逻辑判断；脚本不宜过长，否则阻塞。7.0 引入 Function。
- **Pipeline**：客户端将多条命令**批量发送**，减少 RTT；**不保证原子性**，仅是网络优化。

延伸 / 易错点

- **与 ACID 的关系**：保证隔离性，原子性不完整（无回滚），持久性取决于持久化配置。

28. 典型应用：延迟队列、排行榜、限流

- **延迟队列**：ZSet，score = 执行时间戳，轮询 `ZRANGEBYSCORE key 0 now` 取到期任务，用 `ZREM` 成功与否保证只被一个消费者处理（或用 Lua 原子取出）。
- **排行榜**：ZSet `ZINCRBY` 加分，`ZREVRANGE key 0 9 WITHSCORES` 取 Top10，`ZREVRANK` 查排名。
- **限流**：
  - 固定窗口：`INCR` + `EXPIRE`；
  - 滑动窗口：ZSet 存请求时间戳，删除窗口外记录后计数；
  - 令牌桶：Lua 脚本维护令牌数与上次补充时间。

29. 秒杀场景与防超卖

- **流量分层拦截**：前端静态化 + CDN、按钮置灰防重复提交；网关**限流**；后端用 Redis 承接高并发读写，数据库只处理最终订单。
- **库存预扣（Redis）**：活动前把库存加载到 Redis，用 **Lua 脚本原子执行「判断库存 > 0 → 扣减 → 记录用户已购」**，避免并发超卖与重复购买。
- **异步下单**：扣减成功后发送消息到 **MQ**，消费者异步创建订单、扣减数据库库存，削峰填谷。

- **数据库兜底**：`UPDATE stock SET count = count - 1 WHERE id = ? AND count > 0`（条件更新，影响行数为 0 即失败）；**一人一单**用（用户 ID, 商品 ID）**唯一索引**保证。
- **一致性**：消息消费要**幂等**；下单失败/超时未支付需回补库存；Redis 与数据库库存定期对账。

第二部分 MySQL

一、架构与 InnoDB

30. SELECT 执行流程与 MySQL 架构

- **Server 层**：
  1. **连接器**：TCP 连接、身份认证、权限获取；
  2. **查询缓存**：命中直接返回（**MySQL 8.0 已移除**，因为表一更新就失效）；
  3. **解析器**：词法分析、语法分析，生成语法树；
  4. **预处理/优化器**：检查表和字段是否存在；选择索引、决定 join 顺序，生成执行计划；
  5. **执行器**：校验权限，调用存储引擎接口逐行读取并判断条件。
- **存储引擎层**：负责数据存取，插件式（InnoDB、MyISAM 等）。

延伸 / 易错点

- **长连接问题**：连接占用内存累积，可定期断开或执行 `mysql_reset_connection`。

31. InnoDB vs MyISAM

维度	InnoDB	MyISAM
事务	支持	不支持
锁粒度	**行级锁**	表级锁
外键	支持	不支持
崩溃恢复	支持（redo log）	不支持
索引	**聚簇索引**（数据即索引）	非聚簇，索引存数据地址
MVCC	支持	不支持
`COUNT(*)`	需扫描	保存总行数（无 WHERE 时很快）

- MySQL 5.5 起默认引擎为 InnoDB。

32. InnoDB 存储结构与 Buffer Pool

- 逻辑结构：**表空间 → 段 → 区（1MB，64 个页）→ 页（默认 16KB）→ 行**。页是磁盘与内存交互的**最小单位**。
- 行格式：COMPACT、**DYNAMIC（5.7 起默认）**、COMPRESSED；行记录含变长字段长度列表、NULL 值列表、记录头、隐藏列。
- **Buffer Pool**：缓存数据页和索引页，读写先操作内存，脏页由后台异步刷盘。
  - 改进 LRU：分 **young 区和 old 区**（old 默认占 37%，`innodb_old_blocks_pct`），新页先插入 old 区头部，在 old 区停留超过 `innodb_old_blocks_time`（默认 1000ms）后再次访问才

进 young 区 —— 解决**预读失效**和**全表扫描导致的缓存污染**。
    - 包含 free 链表、flush 链表（脏页）。
- 其他：**Change Buffer**（缓存二级非唯一索引的修改）、**Doublewrite Buffer**（防止页部分写入）、**自适应哈希索引**。

延伸 / 易错点

- **双写缓冲的作用**：16KB 页写到一半宕机（部分写），redo log 无法基于损坏页恢复，先写双写区保证有完整副本。

二、索引

33. B+ 树索引

- 目标：**减少磁盘 IO**，树越矮越好。
- 对比：
    - **B 树**：非叶子节点也存数据，单页能存的键少，树更高；范围查询需中序遍历。
    - **B+ 树**：**非叶子节点只存键和指针，扇出大、树矮**；**所有数据在叶子节点**，叶子之间**双向链表**相连，范围查询高效；查询稳定（都到叶子层）。
    - **哈希**：等值 O(1)，但**不支持范围、排序、最左前缀**。
    - **二叉/红黑树**：树高太高，IO 次数多。
    - **跳表**：层高随数据增长较高，IO 次数多于 B+ 树。
- 估算：主键 BIGINT（8B）+ 页指针（6B）≈ 14B，一个 16KB 非叶子页约存 **1170** 个指针；若一行约 1KB，叶子页存 16 行，**3 层约 1170 × 1170 × 16 ≈ 2000 万行**，根节点常驻内存时只需 1~3 次 IO。

延伸 / 易错点

- **「单表 2000 万」不是硬性规定**：只是按上述行大小估算的经验值，行越大阈值越小。

34. 聚簇索引、二级索引与回表

- **聚簇索引（主键索引）**：叶子节点存**完整行数据**，一张表只有一个。若无主键，选第一个非空唯一索引；都没有则使用隐藏的 6 字节 `DB_ROW_ID`。
- **二级索引（辅助索引）**：叶子节点存**索引列 + 主键值**。
- **回表**：通过二级索引查到主键后，再到聚簇索引中查完整行。

延伸 / 易错点

- **推荐自增主键**：顺序追加写入，**避免页分裂**和随机 IO；主键短，二级索引更省空间。UUID 无序且长，插入易页分裂。自增主键不保证连续（事务回滚、唯一键冲突、批量插入预分配都会产生空洞）。
- **二级索引存主键而非行地址**：行移动（页分裂）时无需更新二级索引。

35. 覆盖索引与索引下推

- **覆盖索引**：查询的列**全部在索引中**，无需回表。EXPLAIN 的 Extra 显示 `Using index`。优化手段：为高频查询建立联合索引覆盖所需列。
- **索引下推 ICP（5.6 引入）**：联合索引遍历时，**在存储引擎层**就用索引中包含的列过滤不满足条件的记录，减少回表次数。Extra 显示 `Using index condition`。
  - 例：索引 (name, age)，`WHERE name LIKE '张%' AND age = 10`，无 ICP 时每条 name 匹配都回表；有 ICP 时先在索引里判断 age。

36. 联合索引与最左前缀原则

- 联合索引 `(a, b, c)` 按 a 排序，a 相同按 b，b 相同按 c，因此**必须从最左列开始连续匹配**才能利用索引定位。
- 可用：`a`、`a,b`、`a,b,c`；`WHERE b=1 AND a=1` 也可（优化器会调整顺序）。
- 不可用（定位）：`b`、`c`、`b,c`。
- `a=1 AND c=3`：只有 a 用于定位，c 可借助 ICP 在索引内过滤。
- **范围查询**：`a > 1 AND b = 2`，a 之后的 b 无法用于定位（a 范围内 b 无序）；但 `>=`、`BETWEEN`、`LIKE 'x%'` 在边界值上 b 仍可部分利用。
- 8.0.13 引入 **索引跳跃扫描（Skip Scan）**，某些条件下缺少最左列也能利用联合索引。

延伸 / 易错点

- **列顺序**：区分度高的列、等值条件列放前面；兼顾覆盖索引与排序需求。

37. 索引失效场景

- 对索引列做**函数或表达式运算**：`WHERE YEAR(create_time)=2024`、`WHERE id+1=10`。
- **隐式类型转换**：字符串列用数字比较 `WHERE phone = 13800000000`（MySQL 将字符串转数字，相当于对列用函数）。反之，整数列和字符串比较不影响索引。
- **左模糊/全模糊**：`LIKE '%abc'`（覆盖索引情况下可能走索引全扫描）。
- **不满足最左前缀**。
- **OR** 条件中有一侧列无索引。
- 优化器根据成本判断**全表扫描更快**（如回表比例很高）。
- `!=`、`NOT IN`、`IS NOT NULL` 等可能因成本原因不走索引（并非一定失效）。

延伸 / 易错点

- **确认是否走索引**：`EXPLAIN` 看 `key`、`type`、`rows`；必要时 `FORCE INDEX` 干预。

38. 建索引原则、唯一索引 vs 普通索引

- 适合：WHERE、JOIN、ORDER BY、GROUP BY 频繁的列；**区分度高**的列；考虑联合索引与覆盖索引。

- 不适合：区分度低（如性别）、频繁更新、数据量很小的表；索引不宜过多（影响写入、占空间）。
- 长字符串用**前缀索引**（`INDEX(email(10))`），但无法用于覆盖索引和排序。
- **唯一 vs 普通**：查询性能几乎无差别；更新时**普通索引可使用 Change Buffer**（不需立即读入页），唯一索引必须读页判断唯一性。业务能保证唯一时，写多读少场景可优先普通索引。

三、事务与隔离级别

39. ACID 及实现

- **原子性 A**：要么全成功要么全回滚 → **undo log**。
- **一致性 C**：数据满足完整性约束，是目的 → 由 A、I、D 共同保证 + 应用层保证。
- **隔离性 I**：并发事务互不干扰 → **锁 + MVCC**。
- **持久性 D**：提交后永久保存 → **redo log**（WAL）。

40. 并发问题与隔离级别

- **脏读**：读到其他事务**未提交**的数据。
- **不可重复读**：同一事务内两次读**同一行**，值不同（被其他事务修改并提交）。
- **幻读**：同一事务内两次**范围查询**，记录**条数**不同（被其他事务插入/删除）。

隔离级别	脏读	不可重复读	幻读
读未提交 RU	可能	可能	可能
读已提交 RC	避免	可能	可能
可重复读 RR（**InnoDB 默认**）	避免	避免	可能（InnoDB 很大程度避免）
串行化	避免	避免	避免

- 实现：RC、RR 基于 **MVCC 的 ReadView**；串行化对读加共享锁。

延伸 / 易错点

- **很多公司用 RC 的原因**：RC 没有间隙锁，**锁冲突和死锁更少**、并发更高；早期 binlog statement 格式下 RC 有主从不一致问题，改用 ROW 格式后可用。

41. RR 级别与幻读

- **快照读**（普通 SELECT）：通过 MVCC，事务内读同一 ReadView，**看不到**其他事务新插入的数据。
- **当前读**（`SELECT ... FOR UPDATE`、UPDATE、DELETE）：通过 **Next-Key Lock（记录锁 + 间隙锁）** 阻止其他事务在范围内插入。

- **没有完全解决**，反例：事务 A 快照读未查到 id=5 → 事务 B 插入 id=5 并提交 → 事务 A `UPDATE ... WHERE id=5` 成功（当前读），该行 trx_id 变为 A 自己 → A 再快照读即可看到 id=5，出现幻读。另外先快照读后当前读也可能读到不同结果。
- 规避：需要时一开始就用 `FOR UPDATE` 加锁。

四、MVCC

42. MVCC 实现原理

- 三要素：**隐藏列 + undo log 版本链 + ReadView**。
- **隐藏列**：`DB_TRX_ID`（最近修改该行的事务 ID）、`DB_ROLL_PTR`（回滚指针，指向 undo log 中旧版本）、`DB_ROW_ID`（无主键时）。
- **版本链**：每次修改把旧版本写入 undo log，通过 roll_ptr 串成链表。
- **ReadView** 四个字段：
  - `m_ids`：创建时**活跃（未提交）事务 ID 列表**；
  - `min_trx_id`：m_ids 中最小值；
  - `max_trx_id`：创建时系统**下一个将分配的事务 ID**（不是 m_ids 最大值）；
  - `creator_trx_id`：创建该 ReadView 的事务 ID。
- **可见性判断**（对版本的 trx_id）：
  1. `trx_id == creator_trx_id` → 自己改的，可见；
  2. `trx_id < min_trx_id` → 已提交，可见；
  3. `trx_id >= max_trx_id` → 在 ReadView 之后开启，不可见；
  4. 介于之间：在 m_ids 中 → 未提交，不可见；不在 → 已提交，可见。
  - 不可见则沿版本链找上一个版本继续判断。
- **RC vs RR**：RC **每次 SELECT 都生成新 ReadView**；RR **只在事务中第一次 SELECT 时生成**，之后复用。

延伸 / 易错点

- **只读事务**：不分配真正的事务 ID，首次修改数据时才分配。
- **undo 清理**：没有任何 ReadView 需要该版本时，由 **purge 线程**清理。

43. 快照读与当前读

- **快照读**：普通 `SELECT`，读 MVCC 历史版本，不加锁。
- **当前读**：读**最新已提交版本**并加锁：`SELECT ... FOR UPDATE`（排他锁）、`SELECT ... LOCK IN SHARE MODE` / 8.0 `FOR SHARE`（共享锁）、`INSERT`、`UPDATE`、`DELETE`。

五、锁

44. 锁的分类

- **全局锁**：`FLUSH TABLES WITH READ LOCK`，全库只读，用于全库逻辑备份（InnoDB 更推荐 `mysqldump --single-transaction`，基于 MVCC 不加全局锁）。
- **表级锁**：
  - 表锁：`LOCK TABLES ... READ/WRITE`；
  - **元数据锁 MDL**：增删改查加 MDL 读锁，改表结构加 MDL 写锁；长事务 + DDL 可能导致后续请求全部阻塞；
  - **意向锁 IS/IX**：加行锁前先在表上加意向锁，使加表锁时**无需逐行检查**；意向锁之间互相兼容；
  - **AUTO-INC 锁**：自增 ID 分配（`innodb_autoinc_lock_mode`，8.0 默认 2 为轻量交错模式）。
- **行级锁（InnoDB）**：
  - **Record Lock 记录锁**：锁单条索引记录；
  - **Gap Lock 间隙锁**：锁记录间的间隙（开区间），**只为防止插入**，间隙锁之间**互相兼容**；
  - **Next-Key Lock**：记录锁 + 间隙锁，**左开右闭区间**，RR 下加锁的基本单位；
  - **插入意向锁**：INSERT 前加的特殊间隙锁，与间隙锁冲突。
- 行锁是**加在索引上的**；没有走索引的更新会扫描全表，对每条记录加 next-key lock，相当于锁全表。

延伸 / 易错点

- **S/X 兼容性**：S 与 S 兼容，S 与 X、X 与 X 互斥。

45. RR 下行锁加锁规则

- 原则 1：加锁基本单位是 **next-key lock（前开后闭）**。
- 原则 2：查找过程中**访问到的对象才加锁**。
- 优化 1：**唯一索引等值查询且记录存在**，next-key lock **退化为记录锁**。
- 优化 2：索引上等值查询，向右遍历到**最后一个不满足条件的值**时，next-key lock **退化为间隙锁**。
- 例：唯一索引 id 有 5、10、15，`WHERE id = 7 FOR UPDATE` → 锁间隙 (5, 10)。
- 覆盖索引下的 `LOCK IN SHARE MODE` 只锁二级索引，不锁主键索引；`FOR UPDATE` 会同时锁主键索引上对应记录。

延伸 / 易错点

- **查看锁**：8.0 用 `performance_schema.data_locks`、`data_lock_waits`。

46. 死锁

- 四个必要条件：**互斥、占有并等待、不可剥夺、循环等待**。
- 常见场景：两个事务以**不同顺序**更新相同行；RR 下两个事务先后对同一间隙加间隙锁（互相兼容），再各自插入，互相等待插入意向锁。
- InnoDB 处理：
  - **死锁检测**（`innodb_deadlock_detect` 默认开启）：等待图检测环，**回滚代价较小（undo 量少）的事务**；
  - **锁等待超时** `innodb_lock_wait_timeout` 默认 **50 秒**。
- 排查：`SHOW ENGINE INNODB STATUS` 查看 `LATEST DETECTED DEADLOCK`。
- 避免：**固定加锁顺序**、**缩短事务**、合理索引减少锁范围、降低隔离级别到 RC、大事务拆分。

47. 乐观锁与悲观锁

- **悲观锁**：假设冲突频繁，操作前加锁，如 `SELECT ... FOR UPDATE`。适合**写多、冲突多**。
- **乐观锁**：假设冲突少，提交时校验，常用**版本号/CAS**：`UPDATE t SET stock=stock-1, version=version+1 WHERE id=1 AND version=#{v}`，影响行数为 0 则重试。适合**读多写少**。
- 库存扣减常用：`UPDATE t SET stock=stock-1 WHERE id=1 AND stock>0`。

六、日志

48. redo log

- InnoDB 引擎层的**物理日志**，记录「某个数据页的某处做了什么修改」。
- **WAL（Write-Ahead Logging）**：修改先写 redo log 再异步刷脏页；redo log 是**顺序写**，比随机写数据页快得多。
- 作用：**崩溃恢复（crash-safe）**，保证持久性。
- 结构：传统为固定大小的一组文件**循环写**，`write pos` 为写入位置，`checkpoint` 为已刷盘可擦除位置，追上时需先刷脏页推进 checkpoint（8.0.30 起由 `innodb_redo_log_capacity` 统一配置）。
- 刷盘策略 `innodb_flush_log_at_trx_commit`：
  - **1（默认）**：每次提交都写入并 fsync，最安全；
  - 0：每秒写入并刷盘，宕机可能丢 1 秒；
  - 2：每次提交写入 OS page cache，每秒 fsync，MySQL 崩溃不丢、操作系统崩溃可能丢。

49. undo log

- **逻辑日志**，记录修改前的数据（insert 记主键用于删除；delete/update 记旧值）。
- 作用：**事务回滚（原子性）** + **MVCC 版本链**。

- 已提交事务的 undo 在不再被 ReadView 需要后由 purge 线程清理；**长事务会导致 undo 无法清理、空间膨胀**。
- undo log 本身的修改也会记录 redo log。

50. binlog 及与 redo log 的区别

- binlog 是 **Server 层的逻辑日志**，记录所有修改数据和表结构的操作，用于**主从复制和数据恢复**。
- 格式：
  - `STATEMENT`：记录 SQL，体积小，但 `NOW()`、`UUID()` 等可能导致主从不一致；
  - `ROW`（**5.7.7 起默认**）：记录行变化前后的值，准确但体积大；
  - `MIXED`：自动选择。
- 刷盘 `sync_binlog`：0 交给 OS；**1 每次提交 fsync（8.0 默认）**；N 每 N 个事务 fsync。

对比	redo log	binlog
层级	InnoDB 引擎层	Server 层，所有引擎可用
类型	物理日志（页修改）	逻辑日志（SQL 或行变化）
写入	循环写，空间固定	**追加写**，写满换新文件
用途	崩溃恢复	主从复制、按时间点恢复

延伸 / 易错点

- **为什么两种日志都需要**：binlog 不记录数据页刷盘状态，无法用于崩溃恢复；redo log 循环写会覆盖，无法做历史归档和主从复制。

51. 两阶段提交

- 目的：保证 **redo log 和 binlog 的逻辑一致**，避免主从数据不一致或恢复后数据不一致。
- 流程（内部 XA 事务）：
  1. **prepare 阶段**：写 redo log，状态标记为 prepare，刷盘；
  2. **写 binlog** 并刷盘；
  3. **commit 阶段**：redo log 标记 commit。
- 崩溃恢复判断：
  - redo 已 commit → 提交；
  - redo 为 prepare：**binlog 完整（能找到对应 XID）→ 提交；否则 → 回滚**。
- 问题：每次提交两次 fsync 性能差 → **组提交（group commit）**，多个事务合并一次刷盘。

52. UPDATE 执行全流程

1. 执行器调用引擎取目标行：所在页在 Buffer Pool 中则直接返回，否则从磁盘读入（加当前读锁）；

2. 记录 **undo log**（旧值），用于回滚和 MVCC；
3. 更新 Buffer Pool 中的数据页，标记为脏页；
4. 写 **redo log**（prepare）；
5. 写 **binlog**；
6. 提交，redo log 置为 commit；
7. 后台线程择机将脏页刷盘。

七、SQL 优化

53. 慢 SQL 定位与 EXPLAIN

- 定位：开启**慢查询日志** `slow_query_log=ON`，`long_query_time`（默认 10 秒，一般调小到 1 秒或更低）；`mysqldumpslow`/`pt-query-digest` 分析；`SHOW PROCESSLIST` 看当前执行。
- 分析：**EXPLAIN** 关键字段：
  - `type`（由好到差）：`system > const > eq_ref > ref > range > index > ALL`，至少达到 range；
  - `key`：实际使用的索引；`key_len`：使用的索引长度（判断联合索引用到几列）；
  - `rows`：预估扫描行数；
  - `Extra`：`Using index`（覆盖索引）、`Using index condition`（ICP）、`Using where`、**`Using filesort`（额外排序）**、**`Using temporary`（临时表）** —— 后两者需重点优化。
- 优化手段：加/调整索引、避免 `SELECT *`、避免索引失效写法、改写子查询为 JOIN、大事务拆分、读写分离、缓存。

54. 深分页优化

- 原因：MySQL 需扫描并**丢弃前 100 万行**（且每行可能回表）。
- 方案：
  - **延迟关联（子查询先查主键）**：`SELECT t.* FROM t JOIN (SELECT id FROM t ORDER BY id LIMIT 1000000, 10) x ON t.id = x.id`，子查询走覆盖索引；
  - **游标/书签分页**：记录上一页最后的 id，`WHERE id > last_id ORDER BY id LIMIT 10`（适合「下一页」场景，不支持随机跳页）；
  - 业务限制最大页数，或使用 ES 等搜索引擎。

55. COUNT 的几种写法

- `COUNT(*)` 与 `COUNT(1)`：统计**所有行**，InnoDB 中处理方式相同、性能一致，官方推荐 `COUNT(*)`。
- `COUNT(主键)`：需读取主键值判断非空。

- `COUNT(普通列)`：**不统计 NULL 值**，需读取该列。
- 性能大致：`COUNT(*) ≈ COUNT(1) > COUNT(主键) > COUNT(列)`。优化器会选择**最小的二级索引**遍历来计数。
- InnoDB 因 MVCC 不同事务可见行数不同，**不能像 MyISAM 那样直接存总行数**；大表计数可用 Redis 计数器或单独计数表。

56. JOIN 与 ORDER BY 优化

- JOIN 算法：
  - **Index Nested-Loop Join**：被驱动表关联字段有索引，逐行查索引，效率高；
  - **Block Nested-Loop Join**：被驱动表无索引，将驱动表数据放入 `join_buffer` 批量比较；**8.0.18 起引入 Hash Join**，8.0.20 起取代 BNL。
- 原则：**小表驱动大表**（「小」指过滤后参与 join 的数据量）；**被驱动表关联字段加索引**；避免过多表 JOIN。
- ORDER BY：
  - 利用索引有序性避免 `filesort`（排序字段与 WHERE 等值字段构成联合索引）；
  - filesort 使用 `sort_buffer`，数据量大时用临时文件归并排序；
  - 区分全字段排序和 rowid 排序（`max_length_for_sort_data`）。

八、主从复制与分库分表

57. 主从复制原理

- 三个线程：
  1. 主库 **binlog dump 线程**：将 binlog 发送给从库；
  2. 从库 **IO 线程**：接收 binlog 写入**中继日志 relay log**；
  3. 从库 **SQL 线程**：读取 relay log **回放**。
- 复制模式：
  - **异步复制**（默认）：主库提交不等从库，可能丢数据；
  - **半同步复制**：至少一个从库**接收并写入 relay log** 后返回 ACK 才响应客户端；5.7 引入 `AFTER_SYNC`（**无损半同步，默认**）：在写 binlog 后、引擎提交前等待 ACK；
  - **组复制 MGR**：基于 Paxos 类协议多数派确认。
- 用途：读写分离、高可用、备份。

58. GTID

- **GTID（全局事务 ID）** = `server_uuid:transaction_id`，每个事务在整个复制集群中唯一。

- 作用：从库通过 `MASTER_AUTO_POSITION=1` **自动定位复制位点**，无需手动指定 binlog 文件名和偏移量；**故障切换**时新主、从库可依据已执行的 GTID 集合自动对齐；天然避免同一事务被重复执行。

59. 主从延迟

- 原因：从库机器性能差、从库查询压力大、**大事务**（回放耗时长）、网络延迟、早期**单线程回放**。
- 并行复制演进：**5.6 按库并行**；**5.7 基于组提交的 LOGICAL_CLOCK 并行**；**8.0 基于 WRITESET 并行**（无冲突事务可并行）。
- 业务应对：
  - 写后立即读的场景**强制读主库**；
  - 判断从库位点/GTID 追上后再读；
  - 关键数据走缓存；
  - 拆分大事务。

60. 分库分表

- 时机：单表数据量过大（常见经验值单表千万级、容量数十 GB）导致查询和 DDL 变慢 → **分表**；单库连接数、QPS、磁盘达到瓶颈 → **分库**。优先考虑索引优化、缓存、读写分离、归档。
- 方式：
  - **垂直分库**：按业务拆分（用户库、订单库）；
  - **垂直分表**：大字段/冷字段拆到扩展表；
  - **水平分表/分库**：按分片键将行分散到多表/多库。
- 分片策略：**Hash 取模**（分布均匀，扩容需迁移）、**Range 范围**（按时间/ID，易扩容但有热点）、一致性哈希、**基因法**（订单 ID 嵌入用户 ID 部分位，便于按两种维度查询）。
- 带来的问题：
  - **全局唯一 ID**：雪花算法（1 位符号 + 41 位毫秒时间戳 + 10 位机器 ID + 12 位序列号，存在**时钟回拨**问题）、号段模式（美团 Leaf）、Redis INCR；
  - **跨库 JOIN**：冗余字段、应用层组装、宽表/ES；
  - **分布式事务**：2PC/XA、TCC、本地消息表、Seata AT、最大努力通知；
  - **跨分片分页、排序、聚合**：归并计算或借助 ES；
  - **扩容迁移**：成倍扩容、双写迁移。
- 中间件：**ShardingSphere（ShardingSphere-JDBC/Proxy）**、MyCat、Vitess（Go 实现，YouTube 开源）。

延伸 / 易错点

- **分片键选择**：选**查询最频繁、分布均匀**的字段（如订单表用 user_id），尽量让查询落在单分片。

61. 分布式事务方案

- **2PC / XA**：协调者 prepare → commit；强一致，但同步阻塞、协调者单点、性能差。
- **TCC**：Try（预留资源）、Confirm（确认）、Cancel（取消）；业务侵入大，需处理**空回滚、幂等、悬挂**。
- **本地消息表**：业务操作和消息写同一本地事务，后台扫描投递 MQ，消费方幂等，最终一致。
- **事务消息**（RocketMQ 半消息）：先发半消息 → 执行本地事务 → 提交/回滚消息，回查机制。
- **Seata AT**：自动生成反向 SQL（undo_log）回滚，基于两阶段与全局锁。
- **Saga**：长事务拆为多个本地事务，失败则执行补偿。

九、SQL 基础与表设计

62. SQL 执行顺序

- 书写顺序：`SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT`。
- **逻辑执行顺序**：`FROM → ON → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT`。
- 推论：WHERE 中**不能使用 SELECT 中定义的别名**和聚合函数（聚合条件放 HAVING）；ORDER BY 可以使用别名（MySQL 在 GROUP BY/HAVING 中也做了别名扩展支持）。

63. 常用数据类型选择

- **CHAR(n) vs VARCHAR(n)**：CHAR 定长（最多 255 字符），不足补空格，适合长度固定的值（如 MD5）；VARCHAR 变长，额外 1~2 字节记录长度。MySQL 5.0+ 中 **n 表示字符数**，单行所有列总长度上限 65535 字节。
- **INT(11) 中的 11**：只是**显示宽度**（配合 ZEROFILL），不影响存储范围；8.0.17 起整数显示宽度已废弃。
- **金额用 DECIMAL**：定点数精确存储，FLOAT/DOUBLE 是二进制浮点数会有精度误差；也可用 BIGINT 存「分」。
- **IP 地址**：可用 `INT UNSIGNED` 配合 `INET_ATON()` / `INET_NTOA()` 存 IPv4，节省空间且便于范围查询；IPv6 可用 `VARBINARY(16)` + `INET6_ATON()`。
- **TEXT 不是无限大**：TEXT 最大 64KB，MEDIUMTEXT 16MB，LONGTEXT 4GB；大字段建议拆表或存对象存储。
- **NULL vs 空字符串**：NULL 表示未知，不占数据空间但需 NULL 值列表标记；`COUNT(列)` 不统计 NULL，`= NULL` 恒不成立需用 `IS NULL`；建议列设 `NOT NULL DEFAULT ''`。

64. DELETE、TRUNCATE、DROP

对比	DELETE	TRUNCATE	DROP
类型	DML	DDL	DDL
作用	删除行（可带 WHERE）	清空全表数据，保留表结构	删除表结构和数据
回滚	可回滚（写 undo）	不可回滚	不可回滚
自增 ID	不重置	重置	—
速度	逐行删除，慢	快（重建表）	快
触发器	触发	不触发	不触发

65. 三大范式与反范式

- **第一范式**：每列都是不可再分的原子值。
- **第二范式**：在 1NF 基础上，非主键列**完全依赖于整个主键**（消除对联合主键的部分依赖）。
- **第三范式**：在 2NF 基础上，非主键列**不传递依赖**于主键（消除非主键列之间的依赖）。
- **反范式**：互联网业务常为减少 JOIN、提升查询性能而**适当冗余字段**，以空间换时间，代价是需要维护数据一致性。

66. 索引分类

- **按数据结构**：B+ 树索引（InnoDB 默认）、哈希索引（Memory 引擎支持；InnoDB 有自适应哈希索引，自动构建）、全文索引（FULLTEXT）。
- **按物理存储**：聚簇索引（主键索引）、二级索引（辅助索引）。
- **按字段特性**：主键索引（唯一且非空，一张表一个）、唯一索引（值唯一，允许多个 NULL）、普通索引、前缀索引。
- **按字段个数**：单列索引、联合索引。
- 索引优缺点：加快查询、排序和分组；但**占用空间**、**降低写入速度**（增删改需维护 B+ 树，可能页分裂）。

67. IN 与 EXISTS、连接查询

- **IN**：先执行子查询得到结果集，再与外表匹配，适合**子查询结果集小**；**EXISTS**：遍历外表，逐行判断子查询是否有结果，适合**外表小、子查询表大且有索引**。新版本优化器会将两者改写为半连接（semi-join），差异变小。
- **NOT IN 陷阱**：子查询结果含 NULL 时 `NOT IN` 返回空集，建议用 `NOT EXISTS` 或 `LEFT JOIN ... IS NULL`。
- **连接类型**：`INNER JOIN` 取交集；`LEFT JOIN` 保留左表全部行，右表无匹配补 NULL；`RIGHT JOIN` 相反；MySQL 不支持 `FULL JOIN`，可用 `UNION` 模拟。
- **避免重复插入**：唯一索引 + `INSERT IGNORE` / `INSERT ... ON DUPLICATE KEY UPDATE` / `REPLACE INTO`（先删后插）。

第三部分 综合实战：Go 秒杀系统

> 本部分把前文 Redis、MySQL 知识点串成一个完整的 Go 秒杀服务：先给出设计文档（按 Google Design Doc 的结构组织），再给出全部代码。代码同时存放在 `/workspace/interview-prep/seckill/`，`go build ./...` 与 `go vet ./...` 均通过。
>
> **知识点标注**：【R-N】表示第一部分 Redis 第 N 个知识点，【M-N】表示第二部分 MySQL 第 N 个知识点（编号全文连续，如【R-29】是「秒杀场景与防超卖」，【M-45】是「RR 下的加锁规则」）。

一、设计文档：Seckill Service

§0 文档信息

项	内容
标题	Seckill Service：基于 Redis 预扣 + MySQL 兜底的 Go 秒杀服务
作者	Lisa
状态	草案（学习用示例，未经生产验证）
评审人	—

§1 Context and Scope（背景与范围）

电商平台需要支持「限时、限量、低价」的秒杀活动：某个商品在约定时刻开售，库存几十到几千件，而同一时刻涌入的请求可能比库存多几个数量级。这类流量有三个特点：

- **读多写少、瞬时峰值**：开售前大量刷新详情页，开售瞬间集中提交下单；
- **绝大多数请求注定失败**：库存远小于请求数，系统价值在于**又快又准地拒绝**；
- **写集中在一行**：所有下单都要扣同一个商品的库存，是典型的**热点行**。

若直接让请求打到 MySQL，`UPDATE ... SET stock = stock - 1` 会在同一行记录锁上排队【M-44】【M-45】，连接池被占满，进而拖垮其他业务。

范围：本服务负责活动查询、秒杀资格校验、库存扣减、订单创建与结果查询。商品管理、支付、物流、用户登录由其他系统负责，本服务只依赖其接口或数据。

现有技术栈：Go 后端，MySQL 8.0（InnoDB）作为业务主库，Redis 6.2+（主从 + 哨兵，或 Cluster）作为缓存。

§2 Goals and Non-Goals（目标与非目标）

Goals

1. **不超卖**：任何情况下，成功订单数 ⩽ 活动库存。
2. **一人一单**：同一用户在同一活动最多一个有效订单。

3. **扛住峰值**：峰值请求由 Redis 与限流承接，MySQL 写入速率只与库存量和消费速度相关，与请求量无关。
4. **数据最终一致**：Redis 预扣结果与 MySQL 订单、库存最终一致，并有对账发现不一致。
5. **可观测、可降级**：关键指标可监控，Redis/MQ 故障时可快速止损。

Non-Goals

- **支付与超时未支付取消**：不在本文范围。后续可用延迟队列（ZSet 或 MQ 延迟消息）实现超时关单并回补库存【R-28】。
- **一单多件、购物车合并下单**：每次固定购买 1 件。
- **验证码、风控模型、设备指纹**：只预留令牌校验入口，具体风控由风控系统负责。
- **少卖为零**：允许极端故障下少卖（库存未卖完），但绝不超卖。这是有意识的取舍，见第 6 节。
- **多活/异地容灾**。

§ 3 Overview（总体方案）

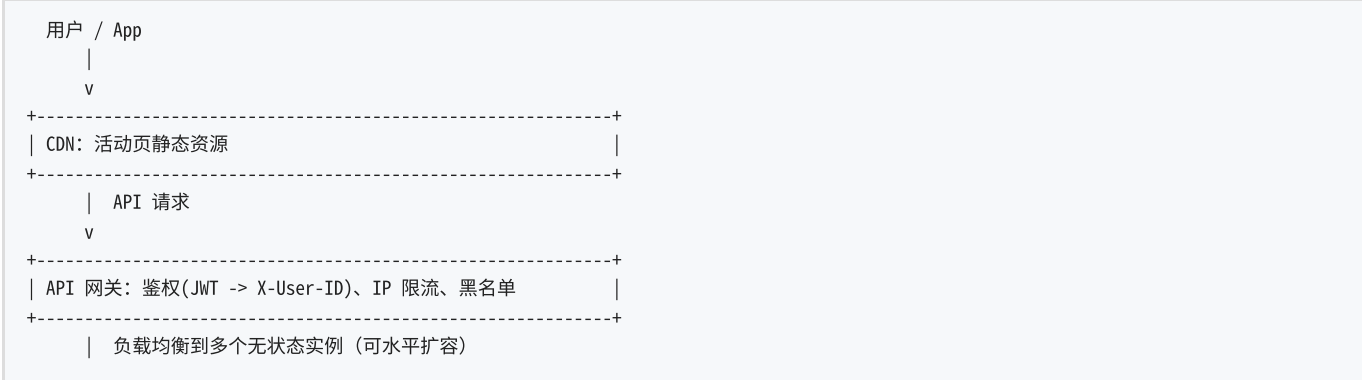
核心思路：**层层削减流量，把并发控制前移到 Redis，把最终正确性交给 MySQL**。

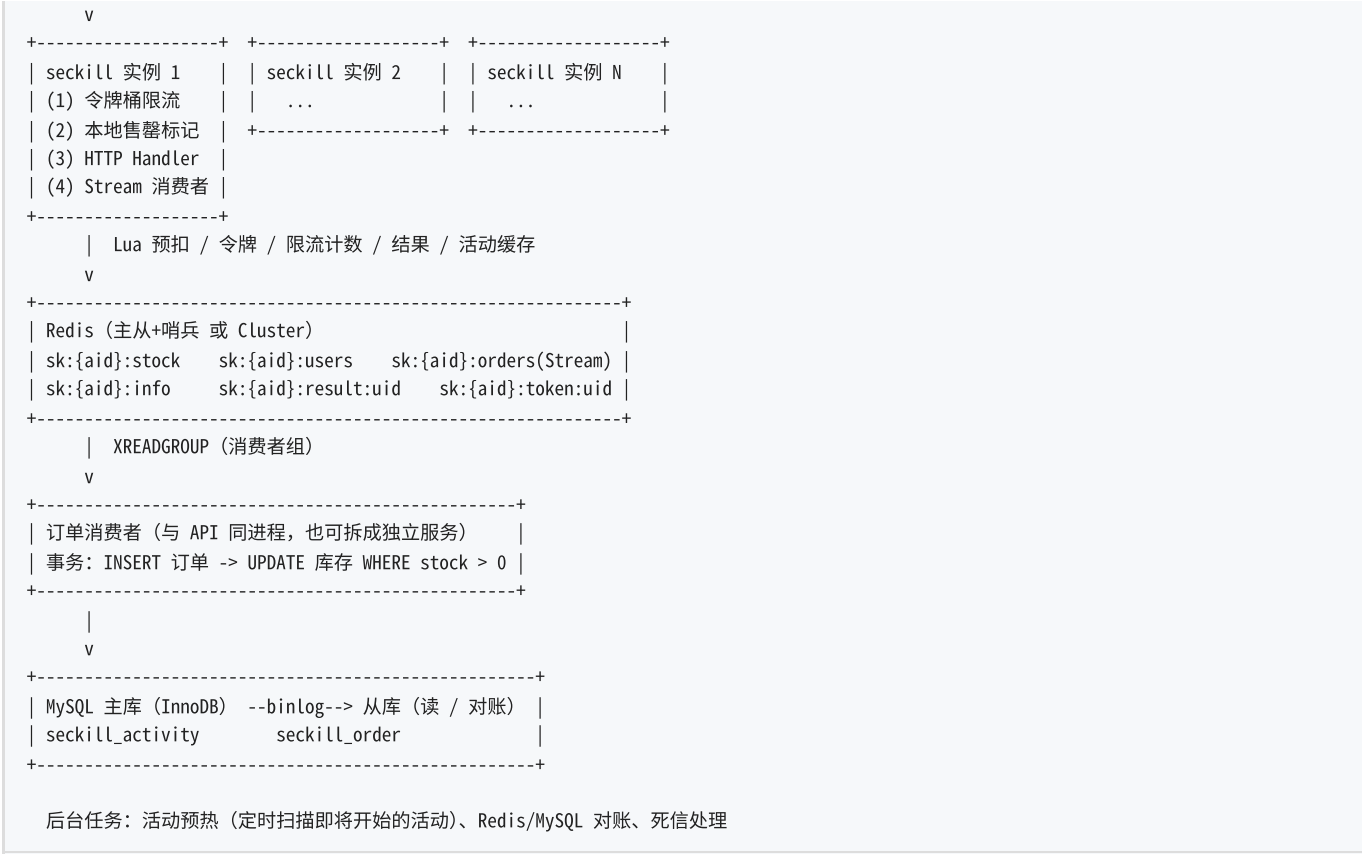
1. **拦截**：CDN/静态页挡掉详情页刷新；网关与实例内令牌桶限流，用户级计数限流【R-28】；
2. **资格**：开售后下发一次性秒杀令牌，令牌用 `GETDEL` 消费，接口地址对脚本不可预知；
3. **预扣**：一段 Lua 脚本原子完成「判重 → 判库存 → 扣减 → 记录已购 → 投递订单消息 → 写排队结果」【R-27】【R-29】；
4. **异步落库**：消费者从 Redis Stream 取消息，在 MySQL 事务中插入订单并做条件扣减【M-39】【M-47】；
5. **兜底与对账**：唯一索引保证一人一单【M-38】，`WHERE stock > 0` 保证不超卖，定时对账 Redis 与 MySQL。

用户感知的流程：点击「抢购」→ 立即返回「排队中」（HTTP 202）→ 前端轮询结果 → 「抢购成功，请支付」或「已售罄」。

§ 4 The Actual Design（详细设计）

4.1 系统架构图





4.2 API

所有接口返回 JSON，用户身份由网关校验 JWT 后以 `X-User-ID` 头注入（示例代码直接读该头）。

方法与路径	说明	成功	主要错误
`GET /api/v1/activities/{id}`	活动详情（走缓存）	200 活动 JSON	404
`POST /api/v1/activities/{id}/token`	开售后获取一次性秒杀令牌（60 s 有效）	200 `{"token"}`	400 未开始/已结束，429
`POST /api/v1/activities/{id}/seckill`	提交秒杀，Body `{"token"}`	**202** `{"status":"queued","order_id"}`	403 令牌无效，409 售罄/重复，429 限流
`GET /api/v1/activities/{id}/result`	轮询结果	200 `{"status":"queued/success/failed/none"}`	—
`POST /admin/activities/{id}/preheat`	手动预热（`X-Admin-Token`）	200	401，404
`GET /debug/vars`	expvar 指标	200	—

设计说明：

- 秒杀接口返回 **202 Accepted** 而不是 201：此时订单只是进入队列，尚未落库。
- 订单号在**预扣时由雪花算法生成**并随消息传递【M-60】，消费者用它做幂等判断，用户也能立即拿到单号。
- 金额字段以字符串返回，与 MySQL `DECIMAL` 对应，避免浮点误差【M-63】。

4.3 数据存储

MySQL 表结构（完整 DDL 见第二节 `sql/schema.sql`）

表	关键字段	索引	设计依据
`seckill_activity`	`id` 自增主键、`price DECIMAL(10,2)`、`total_stock`、`stock INT UNSIGNED`、`start_time`、`end_time`、`status`、`version`	PK(`id`)；`idx_status_start(status, start_time)`	自增主键顺序写入减少页分裂【M-33】【M-34】；预热任务按「状态等值 + 时间范围」查询，符合最左前缀【M-36】；`version` 用于后台编辑的乐观锁【M-47】
`seckill_order`	`id` 雪花 ID、`activity_id`、`user_id`、`goods_id`（冗余）、`price`、`status`	PK(`id`)；**`uk_user_activity(user_id, activity_id)`**；`idx_activity_status(activity_id, status)`	唯一索引兜底一人一单【M-38】【M-66】；`user_id` 在前，同时支撑「我的订单」查询；对账 `COUNT(*)` 可走二级索引覆盖【M-35】【M-55】；冗余 `goods_id` 避免 JOIN（反范式）【M-65】

- 两表均用 InnoDB：需要事务、行锁与崩溃恢复【M-31】【M-48】。
- 雪花 ID 趋势递增，作为主键插入时基本是顺序追加，同时为将来按 `user_id` 分库分表预留全局唯一 ID【M-60】。

Redis Key 设计

Key	类型	内容	TTL	依据
`sk:{aid}:stock`	String	剩余可预扣库存	活动结束 + 1 天	预热时 `SETNX` 写入，防止重复预热覆盖【R-29】
`sk:{aid}:users`	Set	已抢到的用户 ID	同上	`SISMEMBER` O(1) 判重；万级成员时注意大 key【R-3】【R-20】
`sk:{aid}:orders`	Stream	下单消息 `uid/oid/ts`	`MAXLEN ~` 截断	消费者组 + ACK，可靠消息队列【R-3】
`sk:{aid}:info`	String(JSON)	活动详情缓存	10 min + 随机 0~120 s	Cache Aside、TTL 抖动防雪崩【R-19】【R-25】
`sk:{aid}:result:{uid}`	String	`queued:oid` / `success:oid` / `failed:原因`	30 min	供前端轮询，减少查库
`sk:{aid}:token:{uid}`	String	一次性令牌	60 s	`GETDEL` 原子取出并删除
`sk:rl:{uid}:{秒}`	String	用户每秒请求计数	2 s	固定窗口限流【R-28】
`sk:dlq`	Stream	死信消息	人工处理后清理	可靠性兜底

- **Hash Tag**：活动相关 key 都含 `{aid}`，Cluster 只对花括号内的内容计算 CRC16 槽位，所以它们落在同一个哈希槽，多 key 的 Lua 脚本才能执行【R-15】。代价是一个活动的流量集中在一个分片上，见热点讨论（4.4.7）。
- **命名**：`业务:{实体}:属性`，便于 `SCAN` 按前缀排查（禁止 `KEYS`）【R-21】。

4.4 关键流程

4.4.1 活动预热

1. 定时任务每 30 s 查询「已发布、`start_time` ≤ 现在 + 10 min、未结束」的活动（命中 `idx_status_start`）【M-36】；
2. `SETNX sk:{aid}:stock <stock>`，已存在则跳过，**不覆盖进行中的库存**；
3. `XGROUP CREATE sk:{aid}:orders order-creator 0 MKSTREAM`，忽略 `BUSYGROUP`；
4. 每个活动启动一个 Stream 消费协程（同一进程只启动一次）。

预热把库存提前放进内存，开售瞬间就不会出现缓存未命中导致的回源击穿【R-18】。

4.4.2 秒杀主流程（库存预扣 Lua 原子扣减）

```
请求
  |
  v
实例令牌桶 ---------- 拿不到令牌 -----> 429
```

```
  |
  v
本地售罄标记? ----------------- 是 -------------> 409 售罄
  | 否
  v
用户级限流 (Lua INCR+EXPIRE) -- 超限 -----------> 429
  |
  v
活动时间校验 (缓存) ----------- 未开始/已结束 --> 400
  |
  v
GETDEL 令牌校验 --------------- 失败 -----------> 403
  |
  v
雪花算法生成 order_id
  |
  v
+-------------------------------------------------+
| deduct.lua (整段原子执行)                       |
| SISMEMBER users uid  == 1  -> 返回 -2 (重复)    |
| GET stock == nil           -> 返回 -3 (未预热)  |
| stock <= 0                 -> 返回 -1 (售罄)    |
| DECR stock;  SADD users uid                     |
| XADD orders uid oid ts                          |
| SET result:uid "queued:oid" EX ttl              |
| 返回剩余库存                                    |
+-------------------------------------------------+
  |
  v
202 queued + order_id; 剩余为 0 时设置本地售罄标记
```

- **为什么用 Lua**：「判断 + 扣减 + 记录」是**读-判断-写**的复合操作，分成多条命令在并发下会出现两个请求都读到 `stock=1` 的竞态。Redis 单线程执行命令【R-1】，Lua 脚本执行期间不会插入其他命令，相当于一个原子块【R-27】。`MULTI/EXEC` 做不到「根据读到的值决定是否写」，`WATCH` 乐观重试在热点下冲突率极高，所以选 Lua。
- **消息投递放进同一个脚本**：扣减与入队要么都发生、要么都不发生，消除「扣了库存但消息没发出去」的窗口。这是本方案选 Redis Stream 而不是外部 MQ 的主要原因（见第 5 节）。
- **脚本要短**：脚本内只有 O(1) 操作，避免阻塞 Redis【R-22】；go-redis 的 `Script.Run` 先 `EVALSHA`，`NOSCRIPT` 时自动回退 `EVAL`。
- **本地售罄标记**：Redis 返回售罄后，本实例用 `sync.Map` 记住，后续请求连 Redis 都不访问。回滚归还库存时只能清除本实例的标记，其他实例会多拒绝一段时间，这属于「少卖」，可接受；若要精确，可用 Redis Pub/Sub 广播清除。

4.4.3 防超卖（三道防线）

防线	手段	失效场景
1. Redis 预扣	Lua 中 `stock <= 0` 直接拒绝【R-27】【R-29】	主从切换丢失最近写入：主节点异步复制，宕机时新主可能缺少部分 `DECR` 【R-13】【R-14】【R-16】；AOF `everysec` 宕机最多丢约 1 s 写入【R-9】
2. MySQL 条件更新	`UPDATE seckill_activity SET stock = stock - 1 WHERE id = ? AND stock > 0`，影响 0 行即失败	— 这是最终防线
3. 对账	定时比对 Redis 库存 + 已购人数、MySQL 订单数	只发现问题，不阻止

- 条件更新是**当前读**，读的是最新已提交版本并加 X 型记录锁（`id` 为主键等值，RR 下退化为记录锁，不加间隙锁）【M-43】【M-44】【M-45】，所以并发事务对同一行串行执行，不会出现「两个事务都看到 stock=1」。
- 不能用「先 `SELECT stock`（快照读）再 `UPDATE stock = 读到的值 - 1`」：快照读读的是 MVCC 版本【M-42】，并发下会丢失更新。
- `stock` 定义为 `INT UNSIGNED`，即使逻辑出错，减到负数也会报错（严格 SQL 模式下），多一层保险。

4.4.4 一人一单

- **Redis 层**：Lua 内 `SISMEMBER` + `SADD`，与扣减在同一原子块中，并发的两个请求只有一个能通过。
- **MySQL 层**：`UNIQUE KEY uk_user_activity(user_id, activity_id)`；重复插入返回错误码 1062，代码映射为 `ErrDuplicateOrder` 【M-38】【M-67】。
- **不用分布式锁**：按用户加锁（`SET NX PX`）也能做到，但多一次网络往返，还要处理锁过期与误删【R-23】【R-24】；Lua 的 Set 判重已经足够。

4.4.5 异步下单（Redis Stream 消息队列）

```
deduct.lua XADD --> sk:{aid}:orders --XREADGROUP ">"--> 消费者 --> MySQL 事务
                          |                                  |
                          |        成功 / 永久错误: XACK <---+
                          |        临时错误: 不 ACK，留在 PEL
                          v
reclaimLoop: XPENDING(idle > 30s) -> XCLAIM -> 重新处理
             投递次数 >= MaxRetry -> 写 sk:dlq -> 回滚预扣 -> XACK
```

消费者事务（`store.CreateOrder`）：

1. `BEGIN`
2. `INSERT INTO seckill_order ...`：若唯一键冲突（1062）立即返回，**不触碰热点行**；

3. `UPDATE seckill_activity SET stock = stock - 1 WHERE id = ? AND stock > 0`：放在**最后一步**，热点行 X 锁从加锁到提交的时间最短（InnoDB 行锁在事务提交或回滚时才释放，所以越晚加锁、持锁越短）【M-44】【M-46】；
4. `COMMIT`：redo log 与 binlog 通过两阶段提交保证一致，从库与对账读到的数据可信【M-48】【M-50】【M-51】【M-52】。

结果处理（消息至少投递一次，所以消费必须幂等）：

事务结果	含义	处理
成功	正常下单	写结果 `success:oid`，XACK
1062 且已有订单 `id` 等于消息中的 `oid`	同一条消息重复投递	视为成功（幂等），XACK
1062 且订单 `id` 不同	用户已有其他订单（如 Redis 数据丢失后又抢到一次）	回滚预扣并**归还** Redis 库存，结果 `failed:repeat`
影响 0 行	MySQL 已售罄（Redis 与 DB 不一致）	回滚预扣但**不归还**库存，结果 `failed:sold_out`
数据库超时等	临时故障	不 ACK，等待重试；超过 `MaxRetry` 进入死信并回滚

- `rollback.lua` 以 `SREM` 的返回值作为幂等开关：只有用户仍在已购集合中才归还库存，重复回滚不会多加库存。
- 每个活动一个 Stream、一个消费协程：保证 Cluster 下单 key 操作，并且自然把数据库写入速率限制在「消费者数 × 单条事务耗时」，实现**削峰填谷**。

4.4.6 缓存一致性（活动详情）

- **读**：Cache Aside——先读缓存，未命中查库再回写【R-25】。
  - 不存在的活动缓存空值 `<nil>` 1 分钟，防**穿透**【R-17】（活动 ID 量级小，不必上布隆过滤器）；
  - 未命中时用 `singleflight` 合并同一 key 的并发回源，防**击穿**【R-18】；
  - TTL 10 分钟 + 0~120 秒随机抖动，防**雪崩**【R-19】；
  - Redis 故障时直接查库，由前置限流保护 MySQL。
- **写**（后台修改标题/价格）：**先更新数据库，再删除缓存**，删除失败重试 3 次，仍失败靠 TTL 兜底【R-25】。更新时 `WHERE version = ?` 乐观锁，防止两个运营同时编辑互相覆盖【M-47】。
  - 不选「先删缓存再更新库」：并发读会把旧值回填缓存；
  - 进一步可订阅 binlog（如 Canal）异步删除缓存，与业务代码解耦【R-26】【M-50】。
- **库存不走 Cache Aside**：库存是「Redis 为准做预扣、MySQL 为准做最终结算」的两套计数，靠消息与对账保证最终一致，不能用「删缓存」的方式同步，否则会把正在扣减的库存重置。

4.4.7 限流与热点

层级	算法	位置	依据
网关	IP/用户维度限流、黑名单	网关（不在本服务代码中）	—
实例级	令牌桶 `golang.org/x/time/rate`，拿不到令牌立即 429，不排队	进程内存	【R-28】
用户级	固定窗口：`INCR` + 首次 `EXPIRE`，同一 Lua 脚本内执行	Redis	【R-27】【R-28】
下游写入	消费者数量固定，天然限速	Stream 消费	—

- 用户级计数的 INCR 和 EXPIRE 必须原子：分开执行时，若 INCR 后进程崩溃，计数 key 永不过期，该用户被永久限流。
- **热点 key**：单个活动的所有 key 在同一分片，Redis 单分片可承受的 QPS 有限【R-20】。前面的限流与本地售罄标记已过滤绝大部分请求；如仍不够，可把库存拆成 N 个子 key（`sk:{aid#1}:stock` …）分散到不同分片，随机选一个扣减，代价是售罄判断与一人一单需要跨分片处理（见第 5 节）。

4.4.8 对账

每分钟对进行中的活动执行：

- Redis：`GET stock`、`SCARD users`（Pipeline 一次往返）【R-27】；
- MySQL：`SELECT COUNT(*) FROM seckill_order WHERE activity_id = ? AND status <> 2`（走 `idx_activity_status`）【M-55】；
- 期望：`stock + buyers = total_stock`，且 `db_orders ⩽ buyers`（差值是在途消息）。不满足则计数并告警，由人工或补偿任务修复。对账查询可放到从库，注意主从延迟带来的短暂误报【M-57】【M-59】。

§ 5 Alternatives Considered（备选方案）

方案	做法	优点	不选的原因
A. 纯 MySQL 悲观锁	`SELECT ... FOR UPDATE` 后扣减并插入订单	实现简单、强一致	所有请求在热点行锁上排队，连接池耗尽，还可能出现死锁与锁等待超时【M-44】【M-46】【M-47】
B. 纯 MySQL 乐观锁	`UPDATE ... WHERE version = ?` 重试	不持锁等待	热点下冲突率接近 100%，大量无效重试，数据库 QPS 被打满【M-47】
C. Redis 分布式锁 + 普通命令	加锁 → GET → DECR → 释放锁	好理解	每次多 2 次往返，锁串行化后吞吐低；还要处理锁过期、误删、主从切换丢锁【R-23】【R-24】；Lua 一次完成，没有这些问题
D. Kafka / RocketMQ 代替 Stream	预扣成功后由应用发送消息	吞吐与堆积能力更强，RocketMQ 有事务消息、延迟消息	扣减和发消息不在一个原子操作里，进程在两步之间崩溃会产生「扣了库存没订单」，需要事务消息或本地消息表；还要多运维一套集群。流量更大或已有 MQ 平台时应改用（见第 6 节）
E. 同步下单	Lua 预扣成功后立即在请求内写 MySQL	用户立即拿到最终结果，无需轮询	MySQL 写入速率直接暴露给峰值；请求耗时取决于数据库
F. 库存分片	把库存拆到 N 个 key/分片	突破单分片 QPS	一人一单需跨分片判重，售罄判断要汇总；当前规模用不到，作为扩展点
G. MySQL 内合并扣减	把多个扣减请求在应用层合并成一次 `stock = stock - n`	降低热点行锁次数	实现复杂，需要处理部分成功；本方案已经用队列把写入串行化

§ 6 Cross-cutting Concerns（横切关注点）

6.1 安全

- **身份**：只信任网关注入的 `X-User-ID`；服务不应直接暴露公网。示例直接读头，生产应校验网关签名或 mTLS。
- **防脚本**：一次性令牌（`crypto/rand` 16 字节，60 s，`GETDEL` 只能用一次），开售前拿不到令牌；配合网关验证码、风控黑名单。
- **注入**：所有 SQL 使用 `?` 占位符参数化，杜绝 SQL 注入；Lua 参数通过 `KEYS/ARGV` 传入，不拼接脚本。

- **管理接口**：`X-Admin-Token` 用 `subtle.ConstantTimeCompare` 比较，防计时攻击；生产应接入统一权限系统并只在内网开放。
- **输入**：请求体用 `http.MaxBytesReader` 限制 1 KB；路径 ID 校验为正整数；内部错误只返回 `internal error`，细节写日志。

6.2 可靠性

故障	影响	应对
Redis 主节点宕机	哨兵/Cluster 切换期间预扣不可用；异步复制可能丢失最近的扣减与消息【R-13】【R-14】【R-16】	切换期间请求失败（返回 5xx，用户重试）；丢失的扣减由 MySQL 条件更新与唯一索引兜底，**不会超卖**；丢失的消息会造成少卖，由对账发现
Redis 重启	依赖持久化恢复：AOF `everysec` 最多丢约 1 s；推荐开启 RDB+AOF 混合持久化，加快恢复【R-8】【R-9】【R-10】	重新预热时 `SETNX` 不覆盖已有库存；若库存 key 丢失，需人工按「total − DB 订单数」重建，不可直接用初始库存
内存不足被淘汰	秒杀 key 被淘汰会导致数据错乱	秒杀 Redis 独立部署，`maxmemory-policy` 设为 `noeviction` 或 `volatile-*` 并控制其他 key【R-11】【R-12】
MySQL 不可用 / 慢	消费失败	消息留在 PEL，自动重试；超过次数进死信并回滚预扣；用户看到「排队中」直至超时
消费者进程崩溃	已读未 ACK 的消息	其他实例 `XCLAIM` 接管；消费幂等（订单号 + 唯一索引）保证不重复下单
雪花 ID 时钟回拨	ID 重复	回拨 ≤ 5 ms 等待追平，否则报错拒绝；`NODE_ID` 由部署系统保证唯一
实例下线	处理中的请求	`signal.NotifyContext` 收到 SIGTERM 后 `Server.Shutdown` 优雅退出

- 重要取舍：**宁可少卖，不可超卖**。所有不确定情况（Redis 丢数据、MySQL 已售罄）都倾向于拒绝订单，而非放行。

6.3 可观测性

- **指标**（expvar，`GET /debug/vars`，生产可换 Prometheus）：请求数、限流数、售罄数、重复数、入队数、下单成功/失败数、死信数、对账不一致数。
- **需额外监控**：Stream 积压长度（`XLEN` / `XPENDING` 汇总）、Redis 慢查询与内存【R-22】、MySQL 慢 SQL 与行锁等待【M-53】、连接池使用率、主从延迟【M-59】。
- **日志**：`log/slog` JSON 结构化日志，关键字段 `activity`、`uid`、`order_id`、消息 ID，可串联「预扣 → 消息 → 落库」全链路。
- **告警**：死信数 > 0、对账不一致、积压持续增长、429 比例异常。

6.4 性能与容量（估算方法）

- MySQL 写入次数 ≈ 库存量（每件成功商品一次事务），与请求量无关；
- Redis 每个通过限流的请求约 4 次往返（用户限流计数、活动详情缓存、令牌 `GETDEL`、Lua 预扣）；
- 具体容量需按目标 QPS 压测得出，本文不给未经验证的数字。

§ 7 Degree of Constraint（约束程度）

- **强约束**：语言与基础设施固定为 Go + MySQL 8.0 + Redis 6.2+（`GETDEL` 需要 6.2），不引入新的中间件；不超卖、一人一单是硬性业务约束。
- **弱约束**：在这些约束内属于全新服务（greenfield），API、表结构、key 设计可以自由决定；部署形态（API 与消费者同进程或拆分）可以按流量调整。
- **随规模变化的决策**：若库存或流量增长到单分片 Redis 无法承载，或公司已有成熟 MQ 平台，应重新评估方案 D（外部 MQ + 事务消息）与方案 F（库存分片）。

§8 设计点 → 知识点对照

设计点	关联知识点
Lua 原子预扣、脚本要短	【R-1】单线程模型、【R-27】Lua/事务/Pipeline、【R-22】变慢排查、【R-29】秒杀与防超卖
Set 判重、Stream 队列、key 类型选择	【R-3】数据类型、【R-20】大 key / 热 key
Hash Tag 同槽	【R-15】Cluster 哈希槽
活动缓存穿透/击穿/雪崩	【R-17】【R-18】【R-19】
先更新库再删缓存	【R-25】Cache Aside、【R-26】其他读写策略
限流	【R-28】限流、【R-27】Lua
不用分布式锁	【R-23】【R-24】分布式锁
Redis 故障与少卖	【R-8】~【R-14】持久化/淘汰/复制/哨兵、【R-16】脑裂
条件更新防超卖	【M-43】当前读、【M-44】【M-45】锁与加锁规则、【M-42】MVCC
热点行事务顺序	【M-44】行锁、【M-46】缩短事务/死锁
唯一索引一人一单	【M-38】唯一索引、【M-66】索引分类、【M-67】避免重复插入
订单事务与持久性	【M-39】ACID、【M-48】redo、【M-49】undo（回滚）、【M-50】binlog、【M-51】两阶段提交
表与索引设计	【M-31】InnoDB、【M-33】【M-34】B+ 树/聚簇索引、【M-36】最左前缀、【M-63】DECIMAL、【M-65】反范式
雪花 ID	【M-60】分库分表与全局 ID
乐观锁编辑活动	【M-47】乐观/悲观锁
对账	【M-55】COUNT、【M-57】【M-59】主从复制与延迟

二、代码实现

> 代码与仓库根目录 `seckill/` 完全一致（https://github.com/LosAveRoad/interview-prep/tree/main/seckill）。依赖：`github.com/redis/go-redis/v9`、`github.com/go-sql-driver/mysql`（database/sql 驱动）、`golang.org/x/time/rate`、`golang.org/x/sync/singleflight`。只做编译与静态检查，未连接真实数据库运行。

项目结构与使用

```
./cmd/seckill/main.go
./go.mod
./go.sum
./internal/api/handler.go
./internal/cache/cache.go
./internal/cache/keys.go
./internal/cache/lua/deduct.lua
./internal/cache/lua/ratelimit.lua
```

```
./internal/cache/lua/rollback.lua
./internal/config/config.go
./internal/idgen/snowflake.go
./internal/metrics/metrics.go
./internal/model/model.go
./internal/mq/stream.go
./internal/service/seckill.go
./internal/store/mysql.go
./sql/schema.sql
```

- **编译检查**：`cd seckill && go build ./... && go vet ./...`
- **初始化数据库**：`mysql < sql/schema.sql`
- **运行**（需本地 MySQL 与 Redis 6.2+）：`MYSQL_DSN=... REDIS_ADDR=... NODE_ID=1 go run ./cmd/seckill`
- **调用顺序**：预热 `POST /admin/activities/1/preheat` → 获取令牌 `POST /api/v1/activities/1/token` → 秒杀 `POST /api/v1/activities/1/seckill` → 轮询 `GET /api/v1/activities/1/result`（后三者带 `X-User-ID` 头）。

sql/schema.sql

MySQL DDL：两张表与索引（4.3）。

```sql
-- 秒杀系统 MySQL DDL（MySQL 8.0, InnoDB, utf8mb4）
CREATE DATABASE IF NOT EXISTS seckill DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE seckill;

-- 秒杀活动表：一行 = 一个活动（一个商品的一场秒杀）
CREATE TABLE IF NOT EXISTS seckill_activity (
    id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '活动 ID，自增主键，顺序写入避免页分裂',
    goods_id    BIGINT UNSIGNED NOT NULL COMMENT '商品 ID',
    title       VARCHAR(128)    NOT NULL DEFAULT '' COMMENT '活动标题',
    price       DECIMAL(10, 2)  NOT NULL COMMENT '秒杀价，金额用 DECIMAL',
    total_stock INT UNSIGNED    NOT NULL COMMENT '初始库存',
    stock       INT UNSIGNED    NOT NULL COMMENT '数据库剩余库存（最终兜底）',
    start_time  DATETIME        NOT NULL COMMENT '开始时间',
    end_time    DATETIME        NOT NULL COMMENT '结束时间',
    status      TINYINT         NOT NULL DEFAULT 0 COMMENT '0 草稿 1 已发布 2 已下线',
    version     INT UNSIGNED    NOT NULL DEFAULT 0 COMMENT '乐观锁版本号（后台编辑用）',
    created_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_status_start (status, start_time) COMMENT '预热任务按状态 + 开始时间扫描'
) ENGINE = InnoDB COMMENT = '秒杀活动';

-- 秒杀订单表：一人一单由唯一索引兜底
CREATE TABLE IF NOT EXISTS seckill_order (
    id          BIGINT UNSIGNED NOT NULL COMMENT '订单 ID，雪花算法生成，趋势递增',
    activity_id BIGINT UNSIGNED NOT NULL COMMENT '活动 ID',
    user_id     BIGINT UNSIGNED NOT NULL COMMENT '用户 ID',
    goods_id    BIGINT UNSIGNED NOT NULL COMMENT '商品 ID（冗余，减少 JOIN）',
    price       DECIMAL(10, 2)  NOT NULL COMMENT '成交价',
    status      TINYINT         NOT NULL DEFAULT 0 COMMENT '0 待支付 1 已支付 2 已取消',
    created_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uk_user_activity (user_id, activity_id) COMMENT '一人一单；user_id 在前，兼顾「我的订单」查询',
    KEY idx_activity_status (activity_id, status) COMMENT '对账、按活动统计'
) ENGINE = InnoDB COMMENT = '秒杀订单';
```

internal/cache/lua/deduct.lua

库存预扣 + 一人一单 + 入队，原子执行（4.4.2）。

```
-- 库存预扣 + 一人一单 + 投递订单消息，整段脚本在 Redis 中原子执行。
-- 所有 KEY 使用同一个 hash tag {activityID}，保证 Redis Cluster 下落在同一个 slot。
-- KEYS[1] 库存        sk:{aid}:stock   (string, 整数)
-- KEYS[2] 已购用户    sk:{aid}:users   (set)
-- KEYS[3] 订单消息流  sk:{aid}:orders  (stream)
-- KEYS[4] 抢购结果    sk:{aid}:result:{uid} (string)
-- ARGV[1] user_id  ARGV[2] order_id  ARGV[3] 请求时间戳(ms)  ARGV[4] 结果 TTL(秒)
-- 返回: >=0 扣减后剩余库存; -1 已售罄; -2 重复购买; -3 活动未预热
if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then
    return -2
end
local stock = redis.call('GET', KEYS[1])
if not stock then
    return -3
end
stock = tonumber(stock)
if stock <= 0 then
    return -1
end
redis.call('DECR', KEYS[1])
redis.call('SADD', KEYS[2], ARGV[1])
redis.call('XADD', KEYS[3], 'MAXLEN', '~', '1000000', '*',
    'uid', ARGV[1], 'oid', ARGV[2], 'ts', ARGV[3])
redis.call('SET', KEYS[4], 'queued:' .. ARGV[2], 'EX', ARGV[4])
return stock - 1
```

internal/cache/lua/rollback.lua

幂等回滚预扣（4.4.5）。

```
-- 下单失败时回滚预扣: 幂等，只有用户仍在已购集合中才回滚。
-- KEYS[1] 库存  KEYS[2] 已购用户  KEYS[3] 抢购结果
-- ARGV[1] user_id  ARGV[2] 结果值  ARGV[3] 结果 TTL(秒)  ARGV[4] 是否归还库存(1/0)
if redis.call('SREM', KEYS[2], ARGV[1]) == 1 then
    if ARGV[4] == '1' then
        redis.call('INCR', KEYS[1])
    end
    redis.call('SET', KEYS[3], ARGV[2], 'EX', ARGV[3])
    return 1
end
return 0
```

internal/cache/lua/ratelimit.lua

用户级固定窗口限流（4.4.7）。

```
-- 固定窗口限流: INCR 与 EXPIRE 在同一脚本内执行，避免产生没有过期时间的计数 key。
-- KEYS[1] 计数 key  ARGV[1] 窗口内最大请求数  ARGV[2] 窗口长度(秒)
-- 返回 1 放行, 0 拒绝
local n = redis.call('INCR', KEYS[1])
if n == 1 then
    redis.call('EXPIRE', KEYS[1], ARGV[2])
end
if n > tonumber(ARGV[1]) then
    return 0
end
return 1
```

internal/cache/keys.go

Redis key 与 Hash Tag（4.3）。

```go
package cache

import "fmt"

// Redis key 设计：业务前缀 sk + hash tag {activityID}。
// 同一活动的库存、已购集合、消息流、结果放在同一个 slot，Lua 脚本才能在 Cluster 模式下执行。

// StockKey 预扣库存（String）。
func StockKey(aid int64) string { return fmt.Sprintf("sk:{%d}:stock", aid) }

// UsersKey 已抢到的用户集合（Set），实现一人一单。
func UsersKey(aid int64) string { return fmt.Sprintf("sk:{%d}:users", aid) }

// StreamKey 订单消息流（Stream），每个活动一个，避免跨 slot。
func StreamKey(aid int64) string { return fmt.Sprintf("sk:{%d}:orders", aid) }

// InfoKey 活动详情缓存（String/JSON），Cache Aside。
func InfoKey(aid int64) string { return fmt.Sprintf("sk:{%d}:info", aid) }

// ResultKey 用户抢购结果（String），供前端轮询。
func ResultKey(aid, uid int64) string { return fmt.Sprintf("sk:{%d}:result:%d", aid, uid) }

// TokenKey 秒杀令牌（String），隐藏真实秒杀接口、一次性使用。
func TokenKey(aid, uid int64) string { return fmt.Sprintf("sk:{%d}:token:%d", aid, uid) }

// UserRateKey 用户级限流计数（String），按秒分窗口。
func UserRateKey(uid, sec int64) string { return fmt.Sprintf("sk:rl:%d:%d", uid, sec) }

// DeadLetterKey 死信流，记录多次重试仍失败的下单消息。
const DeadLetterKey = "sk:dlq"
```

internal/cache/cache.go

go-redis 封装：预热、预扣、回滚、令牌、Cache Aside（4.4.1 / 4.4.2 / 4.4.6）。

```go
// Package cache 封装秒杀相关的 Redis 操作。
package cache

import (
    "context"
    _ "embed"
    "encoding/json"
    "errors"
    "math/rand/v2"
    "strconv"
    "strings"
    "time"

    "github.com/redis/go-redis/v9"
    "golang.org/x/sync/singleflight"

    "github.com/lisa/seckill/internal/model"
)

var (
    //go:embed lua/deduct.lua
    deductSrc string
    //go:embed lua/rollback.lua
    rollbackSrc string
    //go:embed lua/ratelimit.lua
    rateLimitSrc string
```

```
    // redis.Script 先用 EVALSHA 执行，脚本未缓存（NOSCRIPT）时自动回退到 EVAL。
    deductScript    = redis.NewScript(deductSrc)
    rollbackScript  = redis.NewScript(rollbackSrc)
    rateLimitScript = redis.NewScript(rateLimitSrc)
)

// Deduct 的返回码。
const (
    DeductSoldOut      int64 = -1
    DeductRepeat       int64 = -2
    DeductNotPreheated int64 = -3
)

// emptyPlaceholder 缓存空值，防止缓存穿透。
const emptyPlaceholder = "<nil>"

// ErrActivityNotFound 表示活动不存在（含命中空值缓存）。
var ErrActivityNotFound = errors.New("cache: activity not found")

// Cache 是 Redis 访问层。
type Cache struct {
    rdb       redis.UniversalClient
    sf        singleflight.Group
    resultTTL time.Duration
}

// New 创建 Cache。UniversalClient 兼容单机、哨兵与 Cluster 模式。
func New(rdb redis.UniversalClient, resultTTL time.Duration) *Cache {
    return &Cache{rdb: rdb, resultTTL: resultTTL}
}

// Client 返回底层客户端（供消息队列使用）。
func (c *Cache) Client() redis.UniversalClient { return c.rdb }

// Preheat 把活动库存加载到 Redis。使用 SETNX：活动进行中重复预热不会覆盖已扣减的库存。
func (c *Cache) Preheat(ctx context.Context, a *model.Activity) (bool, error) {
    ttl := time.Until(a.EndTime) + 24*time.Hour // 活动结束后再保留一天用于对账
    return c.rdb.SetNX(ctx, StockKey(a.ID), a.Stock, ttl).Result()
}

// Deduct 执行库存预扣 Lua 脚本，返回剩余库存或负数返回码。
func (c *Cache) Deduct(ctx context.Context, aid, uid, oid int64) (int64, error) {
    keys := []string{StockKey(aid), UsersKey(aid), StreamKey(aid), ResultKey(aid, uid)}
    return deductScript.Run(ctx, c.rdb, keys,
        uid, oid, time.Now().UnixMilli(), int64(c.resultTTL/time.Second)).Int64()
}

// Rollback 回滚预扣（幂等）。restoreStock=false 用于数据库已售罄、不应再归还库存的场景。
func (c *Cache) Rollback(ctx context.Context, aid, uid int64, result string, restoreStock bool) (bool, error) {
    restore := "0"
    if restoreStock {
        restore = "1"
    }
    keys := []string{StockKey(aid), UsersKey(aid), ResultKey(aid, uid)}
    n, err := rollbackScript.Run(ctx, c.rdb, keys,
        uid, result, int64(c.resultTTL/time.Second), restore).Int64()
    return n == 1, err
}

// AllowUser 用户级限流：每秒最多 limit 次请求。
func (c *Cache) AllowUser(ctx context.Context, uid int64, limit int) (bool, error) {
    key := UserRateKey(uid, time.Now().Unix())
    n, err := rateLimitScript.Run(ctx, c.rdb, []string{key}, limit, 2).Int64()
    return n == 1, err
}

// SetToken 保存一次性秒杀令牌。
func (c *Cache) SetToken(ctx context.Context, aid, uid int64, token string, ttl time.Duration) error {
```

```
    return c.rdb.Set(ctx, TokenKey(aid, uid), token, ttl).Err()
}

// ConsumeToken 原子地取出并删除令牌（GETDEL，Redis 6.2+），保证令牌只能使用一次。
func (c *Cache) ConsumeToken(ctx context.Context, aid, uid int64, token string) (bool, error) {
    v, err := c.rdb.GetDel(ctx, TokenKey(aid, uid)).Result()
    if errors.Is(err, redis.Nil) {
        return false, nil
    }
    if err != nil {
        return false, err
    }
    return v == token, nil
}

// SetResult 写入抢购结果。
func (c *Cache) SetResult(ctx context.Context, aid, uid int64, result string) error {
    return c.rdb.Set(ctx, ResultKey(aid, uid), result, c.resultTTL).Err()
}

// GetResult 读取抢购结果，不存在返回空串。
func (c *Cache) GetResult(ctx context.Context, aid, uid int64) (string, error) {
    v, err := c.rdb.Get(ctx, ResultKey(aid, uid)).Result()
    if errors.Is(err, redis.Nil) {
        return "", nil
    }
    return v, err
}

// GetActivity 按 Cache Aside 读取活动详情：
//   - 命中直接返回；命中空值占位符返回 ErrActivityNotFound（防穿透）；
//   - 未命中时用 singleflight 合并并发回源（防击穿）；
//   - 回写缓存时 TTL 加随机抖动（防雪崩）。
func (c *Cache) GetActivity(ctx context.Context, aid int64,
    load func(context.Context, int64) (*model.Activity, error)) (*model.Activity, error) {

    key := InfoKey(aid)
    if raw, err := c.rdb.Get(ctx, key).Result(); err == nil {
        if raw == emptyPlaceholder {
            return nil, ErrActivityNotFound
        }
        var a model.Activity
        if err := json.Unmarshal([]byte(raw), &a); err == nil {
            return &a, nil
        }
    } else if !errors.Is(err, redis.Nil) {
        // Redis 故障时降级直接查库（由上层限流保护数据库）
        return load(ctx, aid)
    }

    v, err, _ := c.sf.Do(key, func() (any, error) {
        a, err := load(ctx, aid)
        if errors.Is(err, ErrActivityNotFound) {
            _ = c.rdb.Set(ctx, key, emptyPlaceholder, time.Minute).Err()
            return nil, err
        }
        if err != nil {
            return nil, err
        }
        if b, err := json.Marshal(a); err == nil {
            ttl := 10*time.Minute + time.Duration(rand.IntN(120))*time.Second
            _ = c.rdb.Set(ctx, key, b, ttl).Err()
        }
        return a, nil
    })
    if err != nil {
        return nil, err
    }
    return v.(*model.Activity), nil
```

```
}

// InvalidateActivity 删除活动缓存（先更新数据库，再删除缓存）。
func (c *Cache) InvalidateActivity(ctx context.Context, aid int64) error {
    return c.rdb.Del(ctx, InfoKey(aid)).Err()
}

// StockSnapshot 读取 Redis 中的剩余库存与已购人数，用于对账。
func (c *Cache) StockSnapshot(ctx context.Context, aid int64) (stock, buyers int64, err error) {
    pipe := c.rdb.Pipeline()
    stockCmd := pipe.Get(ctx, StockKey(aid))
    buyersCmd := pipe.SCard(ctx, UsersKey(aid))
    if _, err = pipe.Exec(ctx); err != nil && !errors.Is(err, redis.Nil) {
        return 0, 0, err
    }
    stock, _ = strconv.ParseInt(stockCmd.Val(), 10, 64)
    return stock, buyersCmd.Val(), nil
}

// ParseResult 解析结果字符串 "queued:<oid>" / "success:<oid>" / "failed:<reason>"。
func ParseResult(v string) (status, detail string) {
    status, detail, _ = strings.Cut(v, ":")
    return status, detail
}
```

internal/store/mysql.go

database/sql：订单事务、条件扣减、唯一键冲突识别（4.4.3 / 4.4.4 / 4.4.5）。

```
// Package store 是 MySQL 访问层（database/sql + go-sql-driver/mysql）。
package store

import (
    "context"
    "database/sql"
    "errors"
    "fmt"
    "time"

    "github.com/go-sql-driver/mysql"

    "github.com/lisa/seckill/internal/model"
)

var (
    // ErrNotFound 记录不存在。
    ErrNotFound = errors.New("store: not found")
    // ErrDuplicateOrder 违反唯一索引 uk_user_activity，即同一用户重复下单。
    ErrDuplicateOrder = errors.New("store: duplicate order")
    // ErrSoldOut 数据库库存已为 0（条件更新影响 0 行）。
    ErrSoldOut = errors.New("store: sold out")
    // ErrVersionConflict 乐观锁版本冲突。
    ErrVersionConflict = errors.New("store: version conflict")
)

// mysqlErrDupEntry 是 MySQL 唯一键冲突错误码。
const mysqlErrDupEntry = 1062

// Store 封装数据库操作。
type Store struct{ db *sql.DB }

// Open 打开连接池并设置池参数。
func Open(dsn string) (*Store, error) {
    db, err := sql.Open("mysql", dsn)
    if err != nil {
        return nil, err
```

```
    }
    db.SetMaxOpenConns(100) // 上限要小于 MySQL max_connections / 实例数
    db.SetMaxIdleConns(50)
    db.SetConnMaxLifetime(30 * time.Minute) // 小于服务端 wait_timeout
    db.SetConnMaxIdleTime(5 * time.Minute)
    return &Store{db: db}, nil
}

// Ping 检查连通性。
func (s *Store) Ping(ctx context.Context) error { return s.db.PingContext(ctx) }

// Close 关闭连接池。
func (s *Store) Close() error { return s.db.Close() }

const activityCols = `id, goods_id, title, price, total_stock, stock, start_time, end_time, status, version`

func scanActivity(row interface{ Scan(...any) error }) (*model.Activity, error) {
    var a model.Activity
    err := row.Scan(&a.ID, &a.GoodsID, &a.Title, &a.Price, &a.TotalStock, &a.Stock,
        &a.StartTime, &a.EndTime, &a.Status, &a.Version)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrNotFound
    }
    return &a, err
}

// GetActivity 按主键查询活动（聚簇索引，一次 B+ 树查找）。
func (s *Store) GetActivity(ctx context.Context, id int64) (*model.Activity, error) {
    row := s.db.QueryRowContext(ctx,
        `SELECT `+activityCols+` FROM seckill_activity WHERE id = ?`, id)
    return scanActivity(row)
}

// ListUpcoming 查询即将开始或进行中的已发布活动，命中索引 idx_status_start。
func (s *Store) ListUpcoming(ctx context.Context, before time.Time) ([]*model.Activity, error) {
    rows, err := s.db.QueryContext(ctx,
        `SELECT `+activityCols+` FROM seckill_activity
         WHERE status = ? AND start_time <= ? AND end_time > NOW()`,
        model.ActivityPublished, before)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var list []*model.Activity
    for rows.Next() {
        a, err := scanActivity(rows)
        if err != nil {
            return nil, err
        }
        list = append(list, a)
    }
    return list, rows.Err()
}

// UpdateActivityInfo 后台修改活动标题/价格，使用版本号乐观锁。
func (s *Store) UpdateActivityInfo(ctx context.Context, a *model.Activity) error {
    res, err := s.db.ExecContext(ctx,
        `UPDATE seckill_activity SET title = ?, price = ?, version = version + 1
         WHERE id = ? AND version = ?`, a.Title, a.Price, a.ID, a.Version)
    if err != nil {
        return err
    }
    if n, _ := res.RowsAffected(); n == 0 {
        return ErrVersionConflict
    }
    return nil
}

// CreateOrder 在一个事务内创建订单并扣减数据库库存。
```

```go
//
// 1. 先 INSERT 订单：唯一索引 uk_user_activity 保证一人一单，冲突立即返回，不碰热点行；
// 2. 后 UPDATE 库存：WHERE stock > 0 的条件更新是防超卖的最后一道防线；
//    UPDATE 会对活动行加 X 型记录锁，放在事务最后执行，持锁时间最短。
func (s *Store) CreateOrder(ctx context.Context, o *model.Order) (err error) {
    tx, err := s.db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    defer func() {
        if err != nil {
            _ = tx.Rollback()
        }
    }()

    if _, err = tx.ExecContext(ctx,
        `INSERT INTO seckill_order (id, activity_id, user_id, goods_id, price, status)
         VALUES (?, ?, ?, ?, ?, ?)`,
        o.ID, o.ActivityID, o.UserID, o.GoodsID, o.Price, model.OrderUnpaid); err != nil {
        var me *mysql.MySQLError
        if errors.As(err, &me) && me.Number == mysqlErrDupEntry {
            return ErrDuplicateOrder
        }
        return fmt.Errorf("insert order: %w", err)
    }

    res, err := tx.ExecContext(ctx,
        `UPDATE seckill_activity SET stock = stock - 1 WHERE id = ? AND stock > 0`, o.ActivityID)
    if err != nil {
        return fmt.Errorf("deduct stock: %w", err)
    }
    n, err := res.RowsAffected()
    if err != nil {
        return err
    }
    if n == 0 {
        err = ErrSoldOut
        return err
    }
    return tx.Commit()
}

// GetOrderByUser 通过唯一索引查询用户在某活动下的订单。
func (s *Store) GetOrderByUser(ctx context.Context, aid, uid int64) (*model.Order, error) {
    var o model.Order
    err := s.db.QueryRowContext(ctx,
        `SELECT id, activity_id, user_id, goods_id, price, status, created_at
         FROM seckill_order WHERE user_id = ? AND activity_id = ?`, uid, aid).
        Scan(&o.ID, &o.ActivityID, &o.UserID, &o.GoodsID, &o.Price, &o.Status, &o.CreatedAt)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrNotFound
    }
    return &o, err
}

// CountOrders 统计活动的有效订单数（对账用，走 idx_activity_status 覆盖索引）。
func (s *Store) CountOrders(ctx context.Context, aid int64) (int64, error) {
    var n int64
    err := s.db.QueryRowContext(ctx,
        `SELECT COUNT(*) FROM seckill_order WHERE activity_id = ? AND status <> ?`,
        aid, model.OrderCanceled).Scan(&n)
    return n, err
}
```

internal/mq/stream.go

Redis Stream 消费者组：ACK、PEL 重试、死信（4.4.5）。

```go
// Package mq 基于 Redis Stream 实现异步下单消息队列（消费者组 + ACK + 重试 + 死信）。
package mq

import (
    "context"
    "errors"
    "log/slog"
    "strconv"
    "strings"
    "time"

    "github.com/redis/go-redis/v9"
)

// OrderMsg 是下单消息。
type OrderMsg struct {
    ActivityID int64
    UserID     int64
    OrderID    int64
    TS         int64 // 用户请求时间（毫秒）
}

// ErrPermanent 包装不可重试的错误（如消息格式错误），消费者直接 ACK。
var ErrPermanent = errors.New("mq: permanent error")

// Handler 处理一条消息。返回 nil 或 ErrPermanent 时 ACK；其他错误保留在 PEL 中等待重试。
type Handler func(ctx context.Context, m OrderMsg) error

// DeadLetterFunc 在消息超过最大投递次数后调用，用于补偿（回滚预扣）。
type DeadLetterFunc func(ctx context.Context, m OrderMsg)

// EnsureGroup 创建消费者组（流不存在时一并创建），已存在则忽略。
func EnsureGroup(ctx context.Context, rdb redis.UniversalClient, stream, group string) error {
    err := rdb.XGroupCreateMkStream(ctx, stream, group, "0").Err()
    if err != nil && !strings.HasPrefix(err.Error(), "BUSYGROUP") {
        return err
    }
    return nil
}

// Consumer 消费一个活动的订单流。
type Consumer struct {
    RDB        redis.UniversalClient
    Stream     string
    Group      string
    Name       string
    ActivityID int64
    MaxRetry   int64
    ClaimIdle  time.Duration
    DLQ        string
    Handle     Handler
    OnDead     DeadLetterFunc
    Log        *slog.Logger
}

// Run 阻塞运行直到 ctx 取消。
func (c *Consumer) Run(ctx context.Context) {
    go c.reclaimLoop(ctx)
    for ctx.Err() == nil {
        streams, err := c.RDB.XReadGroup(ctx, &redis.XReadGroupArgs{
            Group:    c.Group,
            Consumer: c.Name,
            Streams:  []string{c.Stream, ">"}, // ">" 只读取从未投递过的新消息
            Count:    32,
            Block:    2 * time.Second,
        }).Result()
        if errors.Is(err, redis.Nil) || ctx.Err() != nil {
            continue
```

```
            }
            if err != nil {
                c.Log.Error("xreadgroup", "stream", c.Stream, "err", err)
                time.Sleep(time.Second)
                continue
            }
            for _, s := range streams {
                for _, msg := range s.Messages {
                    c.process(ctx, msg)
                }
            }
        }
    }

    func (c *Consumer) process(ctx context.Context, msg redis.XMessage) {
        m, err := c.decode(msg)
        if err == nil {
            err = c.Handle(ctx, m)
        }
        if err != nil && !errors.Is(err, ErrPermanent) {
            // 不 ACK：消息留在 PEL（Pending Entries List），由 reclaimLoop 重新认领
            c.Log.Warn("handle order failed, will retry", "id", msg.ID, "err", err)
            return
        }
        if err != nil {
            c.Log.Error("drop bad message", "id", msg.ID, "err", err)
        }
        if err := c.RDB.XAck(ctx, c.Stream, c.Group, msg.ID).Err(); err != nil {
            c.Log.Error("xack", "id", msg.ID, "err", err)
        }
    }

    // reclaimLoop 周期性检查 PEL：空闲过久的消息重新认领处理；超过最大投递次数的进入死信。
    func (c *Consumer) reclaimLoop(ctx context.Context) {
        t := time.NewTicker(c.ClaimIdle / 2)
        defer t.Stop()
        for {
            select {
            case <-ctx.Done():
                return
            case <-t.C:
            }
            pending, err := c.RDB.XPendingExt(ctx, &redis.XPendingExtArgs{
                Stream: c.Stream, Group: c.Group, Idle: c.ClaimIdle,
                Start: "-", End: "+", Count: 100,
            }).Result()
            if err != nil {
                c.Log.Error("xpending", "err", err)
                continue
            }
            for _, p := range pending {
                msgs, err := c.RDB.XClaim(ctx, &redis.XClaimArgs{
                    Stream: c.Stream, Group: c.Group, Consumer: c.Name,
                    MinIdle: c.ClaimIdle, Messages: []string{p.ID},
                }).Result()
                if err != nil || len(msgs) == 0 {
                    continue // 已被其他消费者认领
                }
                if p.RetryCount >= c.MaxRetry {
                    c.deadLetter(ctx, msgs[0])
                    continue
                }
                c.process(ctx, msgs[0])
            }
        }
    }

    func (c *Consumer) deadLetter(ctx context.Context, msg redis.XMessage) {
        values := map[string]any{"stream": c.Stream, "msg_id": msg.ID}
```

```
    for k, v := range msg.Values {
        values[k] = v
    }
    if err := c.RDB.XAdd(ctx, &redis.XAddArgs{Stream: c.DLQ, Values: values}).Err(); err != nil {
        c.Log.Error("write dlq", "id", msg.ID, "err", err)
        return // 写死信失败则不 ACK，下轮再试
    }
    if m, err := c.decode(msg); err == nil && c.OnDead != nil {
        c.OnDead(ctx, m)
    }
    _ = c.RDB.XAck(ctx, c.Stream, c.Group, msg.ID).Err()
    c.Log.Error("message moved to dead letter queue", "id", msg.ID)
}

func (c *Consumer) decode(msg redis.XMessage) (OrderMsg, error) {
    get := func(k string) (int64, error) {
        s, _ := msg.Values[k].(string)
        n, err := strconv.ParseInt(s, 10, 64)
        if err != nil {
            return 0, errors.Join(ErrPermanent, err)
        }
        return n, nil
    }
    uid, err1 := get("uid")
    oid, err2 := get("oid")
    ts, _ := get("ts")
    if err := errors.Join(err1, err2); err != nil {
        return OrderMsg{}, err
    }
    return OrderMsg{ActivityID: c.ActivityID, UserID: uid, OrderID: oid, TS: ts}, nil
}
```

internal/service/seckill.go

业务编排：秒杀主流程、幂等消费、预热、对账（4.4）。

```
// Package service 实现秒杀核心业务流程。
package service

import (
    "context"
    "crypto/rand"
    "encoding/hex"
    "errors"
    "fmt"
    "log/slog"
    "strconv"
    "sync"
    "time"

    "github.com/lisa/seckill/internal/cache"
    "github.com/lisa/seckill/internal/config"
    "github.com/lisa/seckill/internal/idgen"
    "github.com/lisa/seckill/internal/metrics"
    "github.com/lisa/seckill/internal/model"
    "github.com/lisa/seckill/internal/mq"
    "github.com/lisa/seckill/internal/store"
)

// 业务错误，由 API 层映射为 HTTP 状态码。
var (
    ErrNotFound     = errors.New("activity not found")
    ErrNotStarted   = errors.New("activity not started")
    ErrEnded        = errors.New("activity ended")
    ErrSoldOut      = errors.New("sold out")
    ErrRepeat       = errors.New("already purchased")
```

```
    ErrNotPreheated = errors.New("activity not preheated")
    ErrBadToken     = errors.New("invalid seckill token")
    ErrTooMany      = errors.New("too many requests")
)

// Result 是抢购结果。
type Result struct {
    Status  string `json:"status"` // queued / success / failed / none
    OrderID string `json:"order_id,omitempty"`
    Reason  string `json:"reason,omitempty"`
}

// Service 串联 Redis、MySQL 与消息队列。
type Service struct {
    cfg   config.Config
    cache *cache.Cache
    store *store.Store
    ids   *idgen.Snowflake
    log   *slog.Logger

    soldOut   sync.Map // 本地售罄标记：activityID -> struct{}，售罄后请求不再访问 Redis
    consumers sync.Map // 已启动消费者的活动：activityID -> struct{}
    bgCtx     context.Context
}

// New 创建 Service。bgCtx 控制后台消费者的生命周期。
func New(bgCtx context.Context, cfg config.Config, c *cache.Cache, s *store.Store,
    ids *idgen.Snowflake, log *slog.Logger) *Service {
    return &Service{cfg: cfg, cache: c, store: s, ids: ids, log: log, bgCtx: bgCtx}
}

func (s *Service) loadActivity(ctx context.Context, id int64) (*model.Activity, error) {
    a, err := s.store.GetActivity(ctx, id)
    if errors.Is(err, store.ErrNotFound) {
        return nil, cache.ErrActivityNotFound
    }
    return a, err
}

// Activity 查询活动详情（Cache Aside）。
func (s *Service) Activity(ctx context.Context, id int64) (*model.Activity, error) {
    a, err := s.cache.GetActivity(ctx, id, s.loadActivity)
    if errors.Is(err, cache.ErrActivityNotFound) {
        return nil, ErrNotFound
    }
    return a, err
}

func (s *Service) checkTime(a *model.Activity) error {
    now := time.Now()
    switch {
    case a.Status != model.ActivityPublished:
        return ErrNotFound
    case now.Before(a.StartTime):
        return ErrNotStarted
    case !now.Before(a.EndTime):
        return ErrEnded
    }
    return nil
}

// IssueToken 活动开始后为用户签发一次性令牌（秒杀地址隐藏，防止脚本提前刷接口）。
func (s *Service) IssueToken(ctx context.Context, aid, uid int64) (string, error) {
    a, err := s.Activity(ctx, aid)
    if err != nil {
        return "", err
    }
    if err := s.checkTime(a); err != nil {
        return "", err
```

```
    }
    b := make([]byte, 16)
    if _, err := rand.Read(b); err != nil {
        return "", err
    }
    token := hex.EncodeToString(b)
    return token, s.cache.SetToken(ctx, aid, uid, token, s.cfg.TokenTTL)
}

// Seckill 执行抢购：限流 → 校验 → 令牌 → 本地售罄标记 → Lua 预扣并投递消息。
// 成功只代表“排队中”，订单由消费者异步落库，返回预生成的订单号。
func (s *Service) Seckill(ctx context.Context, aid, uid int64, token string) (int64, error) {
    metrics.Requests.Add(1)

    if _, done := s.soldOut.Load(aid); done {
        metrics.SoldOut.Add(1)
        return 0, ErrSoldOut
    }
    ok, err := s.cache.AllowUser(ctx, uid, s.cfg.UserReqPerSec)
    if err != nil {
        return 0, err
    }
    if !ok {
        metrics.RateLimited.Add(1)
        return 0, ErrTooMany
    }
    a, err := s.Activity(ctx, aid)
    if err != nil {
        return 0, err
    }
    if err := s.checkTime(a); err != nil {
        return 0, err
    }
    valid, err := s.cache.ConsumeToken(ctx, aid, uid, token)
    if err != nil {
        return 0, err
    }
    if !valid {
        return 0, ErrBadToken
    }

    oid, err := s.ids.Next()
    if err != nil {
        return 0, err
    }
    left, err := s.cache.Deduct(ctx, aid, uid, oid)
    if err != nil {
        return 0, err
    }
    switch left {
    case cache.DeductSoldOut:
        s.soldOut.Store(aid, struct{}{})
        metrics.SoldOut.Add(1)
        return 0, ErrSoldOut
    case cache.DeductRepeat:
        metrics.Repeat.Add(1)
        return 0, ErrRepeat
    case cache.DeductNotPreheated:
        return 0, ErrNotPreheated
    }
    if left == 0 {
        s.soldOut.Store(aid, struct{}{})
    }
    metrics.Queued.Add(1)
    return oid, nil
}

// Result 查询抢购结果：先查 Redis，缺失时回查数据库。
func (s *Service) Result(ctx context.Context, aid, uid int64) (Result, error) {
```

```
    v, err := s.cache.GetResult(ctx, aid, uid)
    if err != nil {
        return Result{}, err
    }
    if v != "" {
        status, detail := cache.ParseResult(v)
        if status == "failed" {
            return Result{Status: status, Reason: detail}, nil
        }
        return Result{Status: status, OrderID: detail}, nil
    }
    o, err := s.store.GetOrderByUser(ctx, aid, uid)
    if errors.Is(err, store.ErrNotFound) {
        return Result{Status: "none"}, nil
    }
    if err != nil {
        return Result{}, err
    }
    return Result{Status: "success", OrderID: strconv.FormatInt(o.ID, 10)}, nil
}

// HandleOrder 是消息消费者：在 MySQL 事务中创建订单并扣减库存，保证幂等。
func (s *Service) HandleOrder(ctx context.Context, m mq.OrderMsg) error {
    a, err := s.Activity(ctx, m.ActivityID)
    if errors.Is(err, ErrNotFound) {
        return fmt.Errorf("%w: activity %d", mq.ErrPermanent, m.ActivityID)
    }
    if err != nil {
        return err
    }
    o := &model.Order{ID: m.OrderID, ActivityID: m.ActivityID, UserID: m.UserID,
        GoodsID: a.GoodsID, Price: a.Price}

    err = s.store.CreateOrder(ctx, o)
    switch {
    case err == nil:
        metrics.OrderCreated.Add(1)
        return s.cache.SetResult(ctx, m.ActivityID, m.UserID, "success:"+strconv.FormatInt(m.OrderID, 10))

    case errors.Is(err, store.ErrDuplicateOrder):
        // 消息重复投递（至少一次语义）或历史订单：订单号相同视为已成功，幂等返回
        exist, qerr := s.store.GetOrderByUser(ctx, m.ActivityID, m.UserID)
        if qerr != nil {
            return qerr
        }
        if exist.ID == m.OrderID {
            return s.cache.SetResult(ctx, m.ActivityID, m.UserID, "success:"+strconv.FormatInt(exist.ID, 10))
        }
        // 用户已有其他订单（例如 Redis 数据丢失后重复抢到），本次预扣作废并归还库存
        metrics.OrderFailed.Add(1)
        _, rerr := s.cache.Rollback(ctx, m.ActivityID, m.UserID, "failed:repeat", true)
        s.soldOut.Delete(m.ActivityID)
        return rerr

    case errors.Is(err, store.ErrSoldOut):
        // 数据库兜底发现已售罄（Redis 与 DB 不一致），失败且不归还 Redis 库存
        metrics.OrderFailed.Add(1)
        _, rerr := s.cache.Rollback(ctx, m.ActivityID, m.UserID, "failed:sold_out", false)
        return rerr
    }
    return err // 数据库暂时不可用等，返回错误交给 MQ 重试
}

// OnDeadLetter 消息多次重试仍失败：回滚预扣，让库存回到池子里。
func (s *Service) OnDeadLetter(ctx context.Context, m mq.OrderMsg) {
    metrics.DeadLetters.Add(1)
    if _, err := s.store.GetOrderByUser(ctx, m.ActivityID, m.UserID); err == nil {
        return // 订单其实已落库，无需回滚
    }
```

```
    if _, err := s.cache.Rollback(ctx, m.ActivityID, m.UserID, "failed:system_error", true); err != nil {
        s.log.Error("rollback dead letter", "aid", m.ActivityID, "uid", m.UserID, "err", err)
        return
    }
    s.soldOut.Delete(m.ActivityID)
}

// Preheat 预热即将开始的活动：库存写入 Redis、创建消费者组并启动该活动的消费者。
func (s *Service) Preheat(ctx context.Context, a *model.Activity) error {
    if _, err := s.cache.Preheat(ctx, a); err != nil {
        return err
    }
    stream := cache.StreamKey(a.ID)
    if err := mq.EnsureGroup(ctx, s.cache.Client(), stream, s.cfg.ConsumerGroup); err != nil {
        return err
    }
    if _, loaded := s.consumers.LoadOrStore(a.ID, struct{}{}); !loaded {
        c := &mq.Consumer{
            RDB: s.cache.Client(), Stream: stream, Group: s.cfg.ConsumerGroup,
            Name: s.cfg.ConsumerName, ActivityID: a.ID, MaxRetry: s.cfg.MaxRetry,
            ClaimIdle: s.cfg.ClaimIdle, DLQ: cache.DeadLetterKey,
            Handle: s.HandleOrder, OnDead: s.OnDeadLetter, Log: s.log,
        }
        go c.Run(s.bgCtx)
        s.log.Info("consumer started", "activity", a.ID)
    }
    return nil
}

// PreheatByID 管理接口：手动预热指定活动。
func (s *Service) PreheatByID(ctx context.Context, aid int64) error {
    a, err := s.store.GetActivity(ctx, aid)
    if errors.Is(err, store.ErrNotFound) {
        return ErrNotFound
    }
    if err != nil {
        return err
    }
    return s.Preheat(ctx, a)
}

// PreheatUpcoming 定时任务：预热未来 PreheatIn 内开始的活动。
func (s *Service) PreheatUpcoming(ctx context.Context) {
    list, err := s.store.ListUpcoming(ctx, time.Now().Add(s.cfg.PreheatIn))
    if err != nil {
        s.log.Error("list upcoming", "err", err)
        return
    }
    for _, a := range list {
        if err := s.Preheat(ctx, a); err != nil {
            s.log.Error("preheat", "activity", a.ID, "err", err)
        }
    }
}

// UpdateActivityInfo 后台修改活动：先更新数据库，再删除缓存。
func (s *Service) UpdateActivityInfo(ctx context.Context, a *model.Activity) error {
    if err := s.store.UpdateActivityInfo(ctx, a); err != nil {
        return err
    }
    var err error
    for i := 0; i < 3; i++ { // 删除失败重试；仍失败则依赖 TTL 兜底
        if err = s.cache.InvalidateActivity(ctx, a.ID); err == nil {
            return nil
        }
        time.Sleep(time.Duration(i+1) * 50 * time.Millisecond)
    }
    return err
}
```

```go
// Reconcile 对账：Redis 已购人数应等于 MySQL 订单数 + 在途消息数。
// 这里只做告警，修复由人工或补偿任务完成。
func (s *Service) Reconcile(ctx context.Context) {
    list, err := s.store.ListUpcoming(ctx, time.Now())
    if err != nil {
        s.log.Error("reconcile list", "err", err)
        return
    }
    for _, a := range list {
        stock, buyers, err := s.cache.StockSnapshot(ctx, a.ID)
        if err != nil {
            continue
        }
        orders, err := s.store.CountOrders(ctx, a.ID)
        if err != nil {
            continue
        }
        if stock+buyers != a.TotalStock || orders > buyers {
            metrics.ReconcileDiff.Add(1)
            s.log.Warn("reconcile mismatch", "activity", a.ID, "redis_stock", stock,
                "redis_buyers", buyers, "db_orders", orders, "total", a.TotalStock)
        }
    }
}
```

internal/api/handler.go

HTTP 接口、令牌桶限流中间件、错误码映射（4.2 / 4.4.7）。

```go
// Package api 提供 HTTP 接口（Go 1.22+ ServeMux 路由模式）。
package api

import (
    "crypto/subtle"
    "encoding/json"
    "errors"
    "expvar"
    "log/slog"
    "net/http"
    "strconv"

    "golang.org/x/time/rate"

    "github.com/lisa/seckill/internal/metrics"
    "github.com/lisa/seckill/internal/service"
)

// Handler 持有业务服务与限流器。
type Handler struct {
    svc        *service.Service
    limiter    *rate.Limiter // 单实例令牌桶，保护本实例与下游
    adminToken string
    log        *slog.Logger
}

// New 创建 Handler。
func New(svc *service.Service, qps float64, burst int, adminToken string, log *slog.Logger) *Handler {
    return &Handler{svc: svc, limiter: rate.NewLimiter(rate.Limit(qps), burst),
        adminToken: adminToken, log: log}
}

// Routes 注册路由。
func (h *Handler) Routes() http.Handler {
    mux := http.NewServeMux()
    mux.HandleFunc("GET /api/v1/activities/{id}", h.getActivity)
```

```
    mux.Handle("POST /api/v1/activities/{id}/token", h.rateLimit(http.HandlerFunc(h.issueToken)))
    mux.Handle("POST /api/v1/activities/{id}/seckill", h.rateLimit(http.HandlerFunc(h.seckill)))
    mux.HandleFunc("GET /api/v1/activities/{id}/result", h.result)
    mux.HandleFunc("POST /admin/activities/{id}/preheat", h.preheat)
    mux.Handle("GET /debug/vars", expvar.Handler())
    mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    return mux
}

// rateLimit 令牌桶限流中间件，拿不到令牌立即返回 429（不排队等待）。
func (h *Handler) rateLimit(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if !h.limiter.Allow() {
            metrics.RateLimited.Add(1)
            writeErr(w, http.StatusTooManyRequests, "server busy, please retry")
            return
        }
        next.ServeHTTP(w, r)
    })
}

func (h *Handler) getActivity(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    a, err := h.svc.Activity(r.Context(), aid)
    if err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, a)
}

func (h *Handler) issueToken(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    uid, ok := userID(w, r)
    if !ok {
        return
    }
    token, err := h.svc.IssueToken(r.Context(), aid, uid)
    if err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, map[string]string{"token": token})
}

func (h *Handler) seckill(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    uid, ok := userID(w, r)
    if !ok {
        return
    }
    var req struct {
        Token string `json:"token"`
    }
    if err := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<10)).Decode(&req); err != nil || req.Token == "" {
        writeErr(w, http.StatusBadRequest, "token required")
        return
    }
```

```
    oid, err := h.svc.Seckill(r.Context(), aid, uid, req.Token)
    if err != nil {
        h.fail(w, err)
        return
    }
    // 202 Accepted：请求已受理，订单异步创建，客户端轮询 /result
    writeJSON(w, http.StatusAccepted, map[string]string{
        "status": "queued", "order_id": strconv.FormatInt(oid, 10)})
}

func (h *Handler) result(w http.ResponseWriter, r *http.Request) {
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    uid, ok := userID(w, r)
    if !ok {
        return
    }
    res, err := h.svc.Result(r.Context(), aid, uid)
    if err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, res)
}

func (h *Handler) preheat(w http.ResponseWriter, r *http.Request) {
    got := r.Header.Get("X-Admin-Token")
    if subtle.ConstantTimeCompare([]byte(got), []byte(h.adminToken)) != 1 {
        writeErr(w, http.StatusUnauthorized, "unauthorized")
        return
    }
    aid, ok := pathID(w, r)
    if !ok {
        return
    }
    if err := h.svc.PreheatByID(r.Context(), aid); err != nil {
        h.fail(w, err)
        return
    }
    writeJSON(w, http.StatusOK, map[string]string{"status": "preheated"})
}

// fail 把业务错误映射为 HTTP 状态码；未知错误只记录日志，不向客户端暴露细节。
func (h *Handler) fail(w http.ResponseWriter, err error) {
    switch {
    case errors.Is(err, service.ErrNotFound):
        writeErr(w, http.StatusNotFound, err.Error())
    case errors.Is(err, service.ErrSoldOut), errors.Is(err, service.ErrRepeat):
        writeErr(w, http.StatusConflict, err.Error())
    case errors.Is(err, service.ErrTooMany):
        writeErr(w, http.StatusTooManyRequests, err.Error())
    case errors.Is(err, service.ErrBadToken):
        writeErr(w, http.StatusForbidden, err.Error())
    case errors.Is(err, service.ErrNotStarted), errors.Is(err, service.ErrEnded),
        errors.Is(err, service.ErrNotPreheated):
        writeErr(w, http.StatusBadRequest, err.Error())
    default:
        h.log.Error("internal error", "err", err)
        writeErr(w, http.StatusInternalServerError, "internal error")
    }
}

func pathID(w http.ResponseWriter, r *http.Request) (int64, bool) {
    id, err := strconv.ParseInt(r.PathValue("id"), 10, 64)
    if err != nil || id <= 0 {
        writeErr(w, http.StatusBadRequest, "invalid activity id")
        return 0, false
```

```
    }
    return id, true
}

// userID 演示用：从网关注入的 X-User-ID 头读取用户。生产环境应由网关校验 JWT 后注入。
func userID(w http.ResponseWriter, r *http.Request) (int64, bool) {
    uid, err := strconv.ParseInt(r.Header.Get("X-User-ID"), 10, 64)
    if err != nil || uid <= 0 {
        writeErr(w, http.StatusUnauthorized, "login required")
        return 0, false
    }
    return uid, true
}

func writeJSON(w http.ResponseWriter, code int, v any) {
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    w.WriteHeader(code)
    _ = json.NewEncoder(w).Encode(v)
}

func writeErr(w http.ResponseWriter, code int, msg string) {
    writeJSON(w, code, map[string]string{"error": msg})
}
```

cmd/seckill/main.go

启动、依赖装配、后台任务、优雅退出（6.2）。

```
// Command seckill 启动秒杀服务。
package main

import (
    "context"
    "errors"
    "log/slog"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/redis/go-redis/v9"

    "github.com/lisa/seckill/internal/api"
    "github.com/lisa/seckill/internal/cache"
    "github.com/lisa/seckill/internal/config"
    "github.com/lisa/seckill/internal/idgen"
    "github.com/lisa/seckill/internal/service"
    "github.com/lisa/seckill/internal/store"
)

func main() {
    log := slog.New(slog.NewJSONHandler(os.Stdout, nil))
    if err := run(log); err != nil {
        log.Error("exit", "err", err)
        os.Exit(1)
    }
}

func run(log *slog.Logger) error {
    cfg := config.Load()
    ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
    defer stop()

    db, err := store.Open(cfg.MySQLDSN)
    if err != nil {
```

```
        return err
    }
    defer db.Close()

    // UniversalClient：Addrs 只有一个地址时为单机客户端，多个地址时为 Cluster 客户端
    rdb := redis.NewUniversalClient(&redis.UniversalOptions{
        Addrs:        []string{cfg.RedisAddr},
        Password:     cfg.RedisPassword,
        DB:           cfg.RedisDB,
        PoolSize:     200,
        DialTimeout:  time.Second,
        ReadTimeout:  500 * time.Millisecond,
        WriteTimeout: 500 * time.Millisecond,
    })
    defer rdb.Close()

    pingCtx, cancel := context.WithTimeout(ctx, 3*time.Second)
    defer cancel()
    if err := db.Ping(pingCtx); err != nil {
        return err
    }
    if err := rdb.Ping(pingCtx).Err(); err != nil {
        return err
    }

    ids, err := idgen.New(cfg.NodeID)
    if err != nil {
        return err
    }
    svc := service.New(ctx, cfg, cache.New(rdb, cfg.ResultTTL), db, ids, log)

    // 后台任务：预热即将开始的活动、定期对账
    go every(ctx, 30*time.Second, svc.PreheatUpcoming)
    go every(ctx, time.Minute, svc.Reconcile)

    srv := &http.Server{
        Addr:              cfg.HTTPAddr,
        Handler:           api.New(svc, cfg.GlobalQPS, cfg.GlobalBurst, cfg.AdminToken, log).Routes(),
        ReadHeaderTimeout: 2 * time.Second,
        ReadTimeout:       5 * time.Second,
        WriteTimeout:      5 * time.Second,
        IdleTimeout:       60 * time.Second,
    }
    errCh := make(chan error, 1)
    go func() {
        log.Info("listening", "addr", cfg.HTTPAddr)
        if err := srv.ListenAndServe(); !errors.Is(err, http.ErrServerClosed) {
            errCh <- err
        }
        close(errCh)
    }()

    select {
    case err := <-errCh:
        return err
    case <-ctx.Done():
    }
    // 优雅退出：停止接收新请求，等待处理中的请求完成
    shutdownCtx, cancel2 := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel2()
    return srv.Shutdown(shutdownCtx)
}

// every 立即执行一次 fn，之后每隔 d 执行一次，直到 ctx 取消。
func every(ctx context.Context, d time.Duration, fn func(context.Context)) {
    fn(ctx)
    t := time.NewTicker(d)
    defer t.Stop()
    for {
```

```
            select {
            case <-ctx.Done():
                return
            case <-t.C:
                fn(ctx)
            }
        }
    }
```

internal/config/config.go

配置。

```
    // Package config 从环境变量加载配置，给出适合本地演示的默认值。
    package config

    import (
        "os"
        "strconv"
        "time"
    )

    // Config 是服务运行所需的全部配置。
    type Config struct {
        HTTPAddr      string
        MySQLDSN      string
        RedisAddr     string
        RedisPassword string
        RedisDB       int
        AdminToken    string // 管理接口口令，生产环境应接入统一鉴权

        NodeID int64 // 雪花算法机器号，0~1023，多实例部署时必须唯一

        GlobalQPS     float64 // 单实例令牌桶速率
        GlobalBurst   int     // 单实例令牌桶容量
        UserReqPerSec int     // 单用户每秒最多请求数（Redis 固定窗口）

        ConsumerGroup string
        ConsumerName  string
        MaxRetry      int64         // 消息最大投递次数，超过进入死信
        ClaimIdle     time.Duration // pending 消息空闲多久后被重新认领

        TokenTTL  time.Duration // 秒杀令牌有效期
        ResultTTL time.Duration // 抢购结果保留时间
        PreheatIn time.Duration // 提前多久预热活动
    }

    // Load 读取环境变量。
    func Load() Config {
        host, _ := os.Hostname()
        return Config{
            HTTPAddr:      env("HTTP_ADDR", ":8080"),
            MySQLDSN:      env("MYSQL_DSN", "root:root@tcp(127.0.0.1:3306)/seckill?parseTime=true&loc=Local&charset=utf8mb4"),
            RedisAddr:     env("REDIS_ADDR", "127.0.0.1:6379"),
            RedisPassword: env("REDIS_PASSWORD", ""),
            RedisDB:       envInt("REDIS_DB", 0),
            AdminToken:    env("ADMIN_TOKEN", "change-me"),
            NodeID:        int64(envInt("NODE_ID", 1)),
            GlobalQPS:     float64(envInt("GLOBAL_QPS", 5000)),
            GlobalBurst:   envInt("GLOBAL_BURST", 1000),
            UserReqPerSec: envInt("USER_REQ_PER_SEC", 5),
            ConsumerGroup: env("CONSUMER_GROUP", "order-creator"),
            ConsumerName:  env("CONSUMER_NAME", host),
            MaxRetry:      int64(envInt("MAX_RETRY", 5)),
            ClaimIdle:     30 * time.Second,
            TokenTTL:      60 * time.Second,
```

```
        ResultTTL:     30 * time.Minute,
        PreheatIn:     10 * time.Minute,
    }
}

func env(k, def string) string {
    if v := os.Getenv(k); v != "" {
        return v
    }
    return def
}

func envInt(k string, def int) int {
    if v := os.Getenv(k); v != "" {
        if n, err := strconv.Atoi(v); err == nil {
            return n
        }
    }
    return def
}
```

internal/model/model.go

领域模型。

```
// Package model 定义领域模型。
package model

import "time"

// 活动状态。
const (
    ActivityDraft     int8 = 0
    ActivityPublished int8 = 1
    ActivityOffline   int8 = 2
)

// 订单状态。
const (
    OrderUnpaid   int8 = 0
    OrderPaid     int8 = 1
    OrderCanceled int8 = 2
)

// Activity 对应表 seckill_activity。
type Activity struct {
    ID         int64     `json:"id"`
    GoodsID    int64     `json:"goods_id"`
    Title      string    `json:"title"`
    Price      string    `json:"price"` // DECIMAL 以字符串承载，避免浮点误差
    TotalStock int64     `json:"total_stock"`
    Stock      int64     `json:"stock"`
    StartTime  time.Time `json:"start_time"`
    EndTime    time.Time `json:"end_time"`
    Status     int8      `json:"status"`
    Version    int64     `json:"version"`
}

// Order 对应表 seckill_order。
type Order struct {
    ID         int64     `json:"id"`
    ActivityID int64     `json:"activity_id"`
    UserID     int64     `json:"user_id"`
    GoodsID    int64     `json:"goods_id"`
    Price      string    `json:"price"`
    Status     int8      `json:"status"`
```

```
    CreatedAt  time.Time `json:"created_at"`
}
```

internal/idgen/snowflake.go

雪花算法订单号（4.2）。

```
// Package idgen 实现雪花算法：1 位符号 + 41 位毫秒时间戳 + 10 位机器号 + 12 位序列号。
package idgen

import (
    "errors"
    "sync"
    "time"
)

const (
    nodeBits = 10
    seqBits  = 12
    maxNode  = -1 ^ (-1 << nodeBits)
    seqMask  = -1 ^ (-1 << seqBits)
    // epoch 自定义纪元（2024-01-01 UTC），41 位时间戳约可用 69 年。
    epoch = int64(1704067200000)
)

// ErrClockBackwards 表示检测到时钟回拨。
var ErrClockBackwards = errors.New("idgen: clock moved backwards")

// Snowflake 是并发安全的 ID 生成器。
type Snowflake struct {
    mu     sync.Mutex
    node   int64
    lastMs int64
    seq    int64
}

// New 创建生成器，node 取值 0~1023。
func New(node int64) (*Snowflake, error) {
    if node < 0 || node > maxNode {
        return nil, errors.New("idgen: node id out of range")
    }
    return &Snowflake{node: node}, nil
}

// Next 生成下一个 ID。时钟回拨 5ms 以内等待追平，超过则返回错误。
func (s *Snowflake) Next() (int64, error) {
    s.mu.Lock()
    defer s.mu.Unlock()

    now := time.Now().UnixMilli() - epoch
    if now < s.lastMs {
        if s.lastMs-now > 5 {
            return 0, ErrClockBackwards
        }
        for now < s.lastMs {
            time.Sleep(time.Millisecond)
            now = time.Now().UnixMilli() - epoch
        }
    }
    if now == s.lastMs {
        s.seq = (s.seq + 1) & seqMask
        if s.seq == 0 { // 本毫秒序列号用完，等待下一毫秒
            for now <= s.lastMs {
                now = time.Now().UnixMilli() - epoch
            }
        }
```

```
        } else {
            s.seq = 0
        }
        s.lastMs = now
        return now<<(nodeBits+seqBits) | s.node<<seqBits | s.seq, nil
    }
```

internal/metrics/metrics.go

expvar 指标（6.3）。

```
    // Package metrics 用标准库 expvar 暴露计数器（GET /debug/vars），生产可替换为 Prometheus。
    package metrics

    import "expvar"

    var (
        Requests      = expvar.NewInt("seckill_requests_total")
        RateLimited   = expvar.NewInt("seckill_rate_limited_total")
        SoldOut       = expvar.NewInt("seckill_sold_out_total")
        Repeat        = expvar.NewInt("seckill_repeat_total")
        Queued        = expvar.NewInt("seckill_queued_total")
        OrderCreated  = expvar.NewInt("seckill_order_created_total")
        OrderFailed   = expvar.NewInt("seckill_order_failed_total")
        DeadLetters   = expvar.NewInt("seckill_dead_letter_total")
        ReconcileDiff = expvar.NewInt("seckill_reconcile_mismatch_total")
    )
```

go.mod

模块与依赖。

```
module github.com/lisa/seckill

go 1.26.0

require (
    github.com/go-sql-driver/mysql v1.10.1
    github.com/redis/go-redis/v9 v9.23.0
    golang.org/x/sync v0.24.0
    golang.org/x/time v0.16.0
)

require (
    filippo.io/edwards25519 v1.2.0 // indirect
    github.com/cespare/xxhash/v2 v2.3.0 // indirect
    go.uber.org/atomic v1.12.0 // indirect
    golang.org/x/sys v0.48.0 // indirect
)
```